

CAR DEPOSIT
Premium Real-Time Car Auction Platform



SUBMITTED BY
Mehak Mustafa (2022-ag-8073)

ADVISED BY
Dr. Salman Afsar

A FINAL YEAR PROJECT SUBMITTED FOR THE COURSE
CS/SE/IT-610 FINAL YEAR PROJECT 4(0-4)

DEPARTMENT OF COMPUTER SCIENCE
FACULTY OF SCIENCES
UNIVERSITY OF AGRICULTURE FAISALABAD

DECLARATION

I hereby declare that the contents of the Final Year Project CAR DEPOSIT WEB APP are of my own work and no part has been copied from any published source (except the references). I further declare that this work has not been submitted for award of any other diploma/degree. The university may take action if the information provided is found false at any stage. In case of any default the scholar will be proceeded against as per UAF policy.

Mehak Mustafa

CERTIFICATE

To,
The Controller of Examinations,
University of Agriculture,
Faisalabad.

The committee certify that **Mehak 2022-ag-8073** has successfully completed his/her project in partial fulfillment of the requirements for the course **CS/SE/IT-610 Final Year Project 4(0-4)** Computer Science under our guidance and supervision.

Prof. Dr. Salman Afsar

Dr. Saqib Ali
Chairman,
Department of Computer Science

ACKNOWLEDGEMENT

I thank all who in one way or another contributed in the completion of this report. First, I thank to ALLAH ALMIGHTY, most magnificent and most merciful, for all his blessings. Then I am so grateful to the Department of Computer Science for making it possible for me to study here. My special and heartily thanks to my supervisor, *Dr. Salman Afsar* who encouraged and directed me. His challenges brought this work towards a completion. It is with his supervision that this work came into existence. For any faults I take full responsibility. I am also deeply thankful to my informants. I want to acknowledge and appreciate their help and transparency during my research. I am also so thankful to my fellow students whose challenges and productive critics have provided new ideas to the work. Furthermore, I also thank my family who encouraged me and prayed for me throughout the time of my research. May the Almighty God richly bless all of you

ABSTRACT

The **Car Deposit Web App** is an innovative solution developed to revolutionize the automotive industry by automating and modernizing the traditional vehicle auction process. By integrating cutting-edge web technologies, this app enhances interactivity, speed, and efficiency in digital marketplaces. The platform offers a premium, user-friendly interface that allows users to register, log in securely via JWT or OAuth, and browse luxury vehicle listings. Bids are processed using advanced real-time communication protocols, specifically Socket.io and WebSockets, ensuring instantaneous price updates across all connected clients with sub-500ms latency. The system handles concurrent traffic and race conditions smoothly by utilizing MongoDB database transactions with optimistic locking. In addition to real-time bidding, the app stores high-quality image data via Cloudinary CDN and manages automated user alerts through a two-channel notification system (in-app Bell and Brevo SMTP email). The backend architecture, powered by NestJS and Node.js, supports robust data handling, secure role-based access control, and scalable application performance. The app aims to significantly eliminate bidding friction, minimize conflicting placement errors, and boost overall user engagement for automotive buyers and sellers. By leveraging modern cloud-native design patterns and offering seamless third-party integrations, the Car Deposit App provides a comprehensive digital solution for modernizing automotive sales, making vehicle auctions more secure, data-driven, and highly interactive.

Table of Contents

Chapter 1 - INTRODUCTION	1
1.1 Background:	1
1.2 Description:	1
1.3 Problem Statement:	2
1.4 Scope:	2
1.5 Objectives:	3
1.6 Feasibility:	4
1.7 Requirements:	5
1.7.1 Functional Requirements	5
1.7.2 Non- Functional Requirements	6
1.7.3 Hardware Requirements	8
1.7.4 Software Requirements	9
1.8 Stakeholders:	10
Chapter 2 – MATERIALS & METHODS	5
2.1 Process Model:	5
2.2 Tools & Technologies	6
2.3 Design:	7
2.3.1 Use Case Diagrams:	7
Table 2. 1: Register Account	14
✓ Each use case (e.g., Register, Login, Generate Report) will have a separate scenario like this.	15
✓ Titles must match exactly with the Use Case Diagram.	15
✓ If there's nothing to write under “Unresolved Issues” or “Modification History,” just write None or Version 1.0.	15
✓ Mention "User" or "Admin" under "Authority" based on which actor uses the case.	15
2.3.3 Sequence Diagram:	15
2.3.4 Class Diagram:	19
2.3.5 Data Flow Diagram:	20
2.3.6 ER Diagram:	24
2.3.7 Database Model:	25
Database Model for Car Deposit Platform.	26
2.3.8 Architecture:	27

Chapter 3 - RESULTS & DISCUSSION.....	29
3.1 Testing:	29
3.2 Test Cases:	29
3.2.1 User Login:	29
3.2.2 User Login with Invalid Credentials:.....	30
3.2.3 Forget Password Screen:.....	31
3.2.4 Sigup Screen:	31
3.2.5 View Vehicle Auction Room History:.....	32
3.2.6 Images Screen:	33
3.2.7 Summary Screen:	34
3.2.8 Profile Screen:.....	35
3.2.10 Edit-Profile Screen:.....	36
3.3 Conclusion:	37
Chapter 4 - USER MANUAL.....	38
User Authentication and Log In.....	38
Step-by-Step Log In Instructions	38
User Registration (Sign Up).....	39
Figure 3.1 Signing Up.....	Error! Bookmark not defined.
Step-by-Step Registration Instructions	40
Alternative Quick Registration (Social Sign-Up)	40
Home Page Interface	41
Car Auction Catalog & Search Filter Interface.....	42
Figure 3.4 Car Auction	Error! Bookmark not defined.
Key UI Components & Navigation Controls.....	42
Vehicle Submission Interface ("Sell Your Car")	43
Live Vehicle Auction Detail Interface	44

List of Figures

Figure 1 Stakeholders	11
Figure 2 Agile Activities.....	6
Figure 3 Use Case Diagram	10
Figure 4 Sequence Diagram.....	19
Figure 5 Class Diagram	20
Figure 6 Context Diagram	23
Figure 7 Level 0 DFD	23
Figure 8 Level 1 DFD	24
Figure 9 Entity Relationship Diagram	25
Figure 10 Database Model	26
Figure 11 Application Architecture	28
Figure 12 Singing in.....	38
Figure 13 Singing up.....	39
Figure 14 Home	41
Figure 15 Car Auction	42
Figure 16 Add Car	44
Figure 17 Car Auction Detail.....	45
Figure 18 Profile	46
Figure 19 Report	47
Figure 20 Edit Profile.....	47

Chapter 1 - INTRODUCTION

1.1 Background:

The automotive industry has undergone significant transformation with the growth of digital marketplaces. However, most existing car auction platforms lack real-time interactivity, sophisticated user experience, and modern technical implementation. Traditional auction systems require manual updates, cumbersome interfaces, and periodic price refreshes, creating friction and reducing user engagement.

Car Deposit was conceived as a response to these industry limitations. The platform leverages cutting-edge web technologies including Next.js, NestJS, and Socket.io to deliver a premium real-time car auction experience. Unlike traditional platforms, Car Deposit pushes instant bidding updates to all connected users, ensuring no one misses critical auction events. The implementation demonstrates advanced software architecture patterns, secure authentication mechanisms, and scalable cloud deployment strategies.

This project showcases the integration of modern full-stack development practices, real-time communication protocols, and enterprise-grade security measures. By combining responsive frontend design with a robust microservices-style backend, Car Deposit establishes a solid foundation for a high-end vehicle auction ecosystem.

1.2 Description:

Car Deposit is a comprehensive web-based platform engineered for auctioning premium vehicles. The system is built with a modern, split-stack architecture that separates concerns and enables independent scaling of frontend and backend services.

Key Platform Features:

- **Real-Time Bidding System:** Leverages Socket.io WebSocket protocol to push instantaneous price updates to all connected users, eliminating the need for manual page refreshes.
- **Secure User Authentication:** Implements JWT (JSON Web Tokens) for stateless authentication with support for OAuth-based social login via Google and GitHub.
- **Vehicle Image Management:** Integrates Cloudinary API for cloud-based image hosting, optimization, and delivery with CDN acceleration.
- **Wishlist Functionality:** Allows buyers to bookmark and track their favorite vehicles across multiple sessions.
- **Real-Time Notifications:** Implements both in-app Bell notifications via WebSocket and email notifications via Brevo SMTP service.
- **User Profile Management:** Comprehensive user account management with preference settings and auction history tracking.

- **Responsive Design:** Mobile-first interface optimized for all device types and screen sizes.

The backend architecture is powered by NestJS, a TypeScript-first framework that enables rapid development of scalable server applications. MongoDB Atlas provides flexible document-based storage with automatic scaling. The frontend is built with Next.js and React, offering server-side rendering capabilities, automatic code splitting, and production-grade performance optimization.

1.3 Problem Statement:

Problem 1 - Lack of Real-Time Interactivity: Without instantaneous updates, multiple users may place conflicting bids simultaneously, creating confusion and requiring complex error resolution mechanisms. Users cannot see price changes without manually refreshing the page.

Problem 2 - Limited User Engagement: Traditional platforms lack sophisticated notification systems, leaving sellers unaware of auction activity and buyers missing critical bidding opportunities. This passive information delivery model reduces platform engagement.

Problem 3 - Poor Image Presentation: Basic image hosting limits the ability to showcase high-quality vehicle photography, which is crucial for premium automotive sales.

Problem 4 - Weak Authentication Infrastructure: Many platforms lack robust security measures, exposing user credentials and transaction data to risks. Authentication is often implemented with weak token management and insufficient access control.

Problem 5 - Scalability Limitations: Traditional monolithic architectures struggle to handle concurrent users during high-traffic auction periods, resulting in performance degradation.

Car Deposit addresses all these challenges through modern architecture, real-time protocols, and cloud-native design patterns.

1.4 Scope:

Frontend Development: A fully functional, production-ready web application built with Next.js 14 and React 18, featuring a responsive design that works seamlessly on desktop, tablet, and mobile devices.

Backend Development: A robust RESTful API built with NestJS and Express, implementing business logic, validation, authentication, and real-time communication via Socket.io.

Real-Time Infrastructure: WebSocket-based bidding system capable of handling hundreds of concurrent users with sub-500ms latency for bid updates.

Database Design: MongoDB schema with proper indexing, relationships, and query optimization for efficient data retrieval.

External Integrations: Integration with Cloudinary for image hosting, Brevo for email notifications, and Google/GitHub for OAuth authentication.

Security Implementation: JWT-based authentication, CORS configuration, rate limiting, input validation, and HTTPS/TLS encryption.

Deployment Infrastructure: Cloud deployment on Vercel (frontend) and Render (backend) with MongoDB Atlas for database hosting.

Testing and Quality Assurance: Comprehensive unit tests, integration tests, and end-to-end testing for all critical features.

Documentation: Complete API documentation, deployment guides, and architecture documentation.

1.5 Objectives:

Goal: Automate and modernize the vehicle auction process to improve real-time interactivity, security, and scalability for car buyers, sellers, and administrators.

- **Develop a Premium Frontend Interface:** Build a highly responsive, mobile-first web application using Next.js 14, React 18, and Tailwind CSS for seamless user interaction across all modern devices.
- **Implement Secure Identity and Access Management:** Integrate robust user registration and session management using stateless JWT authentication alongside Google and GitHub OAuth 2.0 social logins via Passport.js.
- **Integrate Real-Time Bidding Infrastructure:** Implement Socket.io WebSocket gateways to handle sub-500ms latency for bidirectional data transmission, pushing instant price updates to all active participants.
- **Enable High-Performance Image Optimization:** Integrate Cloudinary API to handle high-resolution vehicle photography, allowing direct signed uploads and CDN-accelerated multi-size image distribution.
- **Create a Scalable Persistence Layer:** Design a MongoDB Atlas cloud database schema with proper document indexing to efficiently store records of user profiles, bidding histories, and vehicle listings.
- **Deploy a Reliable Two-Channel Notification System:** Develop background job workers to push instantaneous in-app Bell notifications via Socket.io and transactional emails using Brevo SMTP for critical auction events like outbids or wins.
- **Ensure Multi-Tier Architecture Reliability:** Build a scalable, split-stack system deployed on Vercel (frontend) and Render (backend) that supports over 1,000 concurrent users while preventing race conditions through database transactions and optimistic locking.

1.6 Feasibility:

1.6.1 Technical Feasibility

The project leverages highly stable, well-documented, and enterprise-grade technologies, including Next.js 14, NestJS, and Socket.io, ensuring robustness and modern full-stack capabilities. NestJS provides a structured architecture with dependency injection, while Socket.io offers reliable, low-latency WebSocket connections essential for real-time bid broadcasting. Database atomic transactions and optimistic locking mechanisms in MongoDB effectively mitigate race conditions during concurrent bidding spikes. Therefore, the technical implementation is completely feasible with the selected stack.

1.6.2 Schedule Feasibility

The development lifecycle of the platform is designed to fit strictly within the academic timeline by breaking down the requirements into manageable Agile sprints. Early sprints focus on core backend architecture, REST API design, and database normalization. Subsequent sprints deal with WebSocket integration for live bidding, Cloudinary image management, and third-party notification services. The final sprints are dedicated to end-to-end integration testing, performance optimization, and deployment on Vercel and Render, ensuring timely delivery.

1.6.3 Economic Feasibility

The project is highly cost-effective as it relies entirely on open-source frameworks, software packages, and cloud hosting platforms with generous free-tier ecosystems. Utilizing open-source tools like Node.js, Tailwind CSS, and Mongoose drastically eliminates licensing overheads. Furthermore, cloud infrastructure costs are minimized during development by utilizing the free hosting tiers provided by Vercel for the frontend, Render for the backend, and MongoDB Atlas for cloud database hosting.

1.6.4 Legal/Ethical Feasibility

The application strictly complies with modern data privacy regulations, web security standards, and ethical data handling policies. User credentials are systematically protected against data breaches using industrial-grade bcrypt password hashing. Stateless authentication tokens (JWT) are safely transmitted via secure, HTTP-only cookies to prevent cross-site scripting (XSS) and data interception. All communications utilize HTTPS/TLS transport layer encryption to maintain privacy and user trust.

1.6.5 Resource Feasibility

The hardware and software assets required to engineer, run, and test the system are readily available. On the software side, IDEs (like VS Code), container utilities, version control systems (GitHub), and API testing clients are free to use. On the hardware side, a standard development machine running a modern operating system is fully capable of spinning up the localized Node.js environment and handling the test suites, making the project highly resource-feasible.

1.6.6 Operational Feasibility

From an operational perspective, the platform delivers immense practical value by solving the critical friction points of traditional online auctions. By replacing manual page refreshes and slow polling with an automated, real-time WebSocket communication loop, it provides instant clarity to buyers and sellers alike. The inclusion of automated outbid emails and responsive interface controls guarantees that the system is intuitive and operationally efficient for everyday users.

1.7 Requirements:

1.7.1 Functional Requirements

FR01: User Authentication and Social Login

FR01-01	The system shall collect credentials (email and password) from the user for traditional login.
FR01-02	The system shall authenticate credentials against hashed passwords stored in the database using bcrypt.
FR01-03	The system shall integrate third-party OAuth 2.0 (Google and GitHub) to allow users to securely authenticate without a manual password.
FR01-04	The system shall issue secure, stateless JSON Web Tokens (JWT) upon successful authentication and store them in secure, HTTP-only cookies.
FR01-05	If the authentication details are invalid, the system shall reject access and display an appropriate error message.

FR02: User Registration and Profile Management

FR02-01	The system shall allow new users to register by providing their name, email, password, and selecting a primary account role (Buyer or Seller).
FR02-02	The system shall enforce data validation rules during registration (e.g., verifying unique emails and demanding strong password criteria).
FR02-03	The system shall allow authenticated users to update their profile settings, view their active bidding history, and manage listed items.

FR03: Vehicle Listing and Image Management (Sellers)

FR03-01	The system shall allow verified sellers to create new vehicle listings by entering mandatory fields like make, model, year, description, base price, and auction expiration time.
FR03-02	The system shall enable sellers to upload multiple high-resolution images of the vehicle.
FR03-03	The system shall securely route image payloads directly to Cloudinary API using a signed preset, and store the optimized CDN URLs in the database.

FR04: Real-Time Bidding Framework.

FR04-01	The system shall open a persistent WebSocket connection via Socket.io when a user visits a specific vehicle's live auction page.
FR04-02	The system shall validate placed bids to ensure the bid amount exceeds the current highest bid plus the predefined minimum increment value.
FR04-03	The system shall process incoming bids using atomic database transactions and optimistic locking to prevent race conditions from concurrent users.
FR04-04	Upon a successful transaction, the system shall instantaneously broadcast the updated bid value and countdown timer to all active clients in that vehicle's socket room within 500 milliseconds.

FR05: Live Wishlist Management

FR05-01	The system shall allow logged-in buyers to add or remove specific vehicle listings to/from their personal wishlist.
FR05-02	The system shall display a dedicated dashboard aggregating all wishlisted items, complete with live WebSocket-driven price updates.

FR06: Two-Channel Notifications

FR06-01	The system shall trigger real-time, in-app Bell notifications via Socket.io to alert users instantly if they are outbid or if an auction closes.
FR06-02	The system shall queue and send transactional emails via Brevo SMTP to notify buyers when they officially win an auction or when critical security changes occur on their profile.

1.7.2 Non- Functional Requirements

These define the quality attributes and performance of the system:

- **Performance:** The system must broadcast real-time bid updates via Socket.io to all connected users within a maximum latency of 500 milliseconds.
 - Typical database queries (such as fetching active vehicle listings or user profiles) must respond within 100 milliseconds.

- Critical pages on the Next.js frontend must achieve a core web vital largest contentful paint (LCP) score of under 2 seconds on a standard 4G network.
- **Scalability:** The architecture must comfortably support at least 1,000 concurrent active users and handle rapid bidding spikes during the final minutes of high-profile auctions without performance degradation.
 - The backend services on Render must be designed to scale horizontally to accommodate increased data traffic and active WebSocket channels.
- **Reliability:** The platform must handle high-concurrency race conditions gracefully, achieving a 100% success rate in enforcing bid sequence integrity through database-level atomic operations and optimistic locking.
 - Data persistence layers must utilize automated 24-hour backup intervals to prevent data loss or transaction corruption.
- **Usability:** The user interface must feature an intuitive, clutter-free, and mobile-first responsive design using Tailwind CSS, ensuring accessibility for buyers and sellers operating on various screen sizes and device viewports.
 - Form fields, bidding input boxes, and automated navigation elements must include inline validation tooltips and clear error feedback to support less tech-savvy users.
- **Security:** All communications and data in transit between the client browser and the API endpoints must be encrypted using standard HTTPS/TLS protocols.
 - Passwords must never be stored in plain text and must be securely salted and hashed using the bcrypt algorithm before database storage.
 - Identity authorization must rely on stateless JSON Web Tokens (JWT) stored inside secure, HTTP-only, and SameSite cookies to mitigate Cross-Site Scripting (XSS) and Cross-Site Request Forgery (CSRF) vulnerabilities.
- **Availability:** The production deployment across Vercel, Render, and MongoDB Atlas cloud ecosystems must guarantee an operational uptime of 99.9% (excluding scheduled maintenance hours).
- **Portability (Compatibility):** The application must function seamlessly across all modern evergreen web browsers, including Google Chrome, Mozilla Firefox, Apple Safari, and Microsoft Edge.
 - The frontend interface must adapt natively to mainstream operating systems including Windows, macOS, Android, and iOS via progressive responsiveness.

- **Maintainability:** The backend source code must be engineered following NestJS standard modular design patterns (using distinct Modules, Controllers, Services, and Gateways) to facilitate automated unit testing via Jest and ensure frictionless codebase upgrades.

1.7.3 Hardware Requirements

The physical and cloud infrastructure resources required for the development, hosting, and end-user access of the platform are specified below:

For Developers (Local Workstation):

- **Processor:** Intel Core i5 / AMD Ryzen 5 or equivalent (Intel Core i7 recommended for handling concurrent local servers smoothly).
- **RAM:** Minimum 8 GB (16 GB recommended for running Next.js frontend, NestJS backend, and local testing environments simultaneously).
- **Storage:** 256 GB SSD or more (SSD is highly recommended to ensure rapid node_modules indexing and fast project compilation).
- **Network:** Standard broadband internet connection for downloading dependencies, handling Git version control, and interacting with cloud services.

For Deployment:

- **Frontend Hosting (Vercel):** Managed Serverless platform with global CDN acceleration (handles dynamic routing and static asset rendering automatically).
- **Backend Hosting (Render):** Cloud application instance running on standard Linux containers with a minimum of 512 MB to 1 GB RAM for smooth event-loop execution and WebSocket lifecycle handling.
- **Database Cluster (MongoDB Atlas):** Shared cloud cluster tier (M0 or M10 cluster) with elastic SSD storage to efficiently manage document collections and session states.

For Users:

- **Devices:** Any modern computing device such as a Smartphone, Tablet, Laptop, or Desktop PC.
- **RAM:** Minimum 2 GB RAM on mobile devices or computers to handle real-time DOM updates and WebSockets efficiently.
- **Display Resolution:** Minimum 320px width resolution (fully optimized via responsive Tailwind CSS grids for mobile viewports up to 4K desktop screens).

- **Network Connectivity:** A stable 3G, 4G, 5G, or Wi-Fi internet connection to ensure sub-500ms real-time bid sync without socket disconnections.

1.7.4 Software Requirements

The software utilities, development frameworks, and platforms necessary to develop, maintain, and run the system are detailed below:

Operating Systems:

- **Development Environment:** Windows 10/11, macOS, or any mainstream Linux distribution (Ubuntu/Debian).
- **Production Deployment Environment:** Linux (Ubuntu/Alpine containers managed automatically within Vercel and Render environments).
- **Client Interface:** Any modern OS running a compatible web browser (Windows, macOS, iOS, Android).

Development Frameworks and Runtimes:

- **Frontend Architecture:** Next.js 14 (React 18 framework) leveraging Server-Side Rendering (SSR) and static generation.
- **Backend Architecture:** NestJS (Enterprise-grade Node.js progressive framework built on top of Express).
- **Real-Time Layer:** Socket.io (Engine.IO implementation for reliable WebSocket connectivity).

Programming and Scripting Languages:

- **Primary Language:** TypeScript (used across both frontend and backend for strong compilation-time type safety).
- **Styling Engine:** Tailwind CSS (utility-first CSS framework for design implementation).
- **Runtime Environment:** Node.js (v18.x or higher).

Database and Object Modeling Utilities:

- **Database Management System:** MongoDB Atlas (Cloud-hosted NoSQL document database).
- **Object Data Modeling (ODM):** Mongoose library (for schema validation and database connectivity within NestJS).

Version Control and Collaboration:

- **Version Control System:** Git (Local version tracking).
- **Collaboration & CI/CD Pipeline:** GitHub (for remote repository hosting and automated deployment hooks).

Cloud Platforms and External Services:

- **Frontend Hosting:** Vercel Platform (with edge network optimizations).
- **Backend Application Service:** Render Cloud Services.
- **Image Hosting CDN:** Cloudinary API & Cloud Service.
- **Email Communication Gateway:** Brevo SMTP API (formerly Sendinblue).

1.8 Stakeholders:

The individuals, groups, or organizations that have an active interest in the development, deployment, operation, and success of the platform are detailed below:

- **Buyers (Bidders):** The primary end-users who interact with the frontend interface to browse premium vehicle listings, add cars to their live wishlists, receive instant outbid alerts, and place real-time bids via WebSockets.
- **Sellers (Dealerships/Individual Owners):** Key users who utilize the platform to register accounts, create comprehensive vehicle listings, upload high-resolution vehicle media directly to the cloud CDN, and monitor active bidding activity on their assets.
- **Platform Administrators (Admins):** Internal stakeholders responsible for regulating the marketplace. They monitor active auctions, manage user account verifications, resolve bidding disputes, oversee system health, and manage data configuration.
- **Project Development Team:**
 - **Backend Developers:** Tasked with building the robust microservices-style business logic, designing REST APIs with NestJS, and creating Socket.io gateways for live bid synchronization.
 - **Frontend Developers:** Responsible for engineering the highly interactive, responsive, and server-side rendered user interface using Next.js, React, and Tailwind CSS.
 - **UI/UX Designers:** Responsible for crafting wireframes, modern user journeys, and premium visual interfaces optimized for desktop and mobile viewports.
- **Cloud Infrastructure & Service Providers:** Third-party tech entities critical to operations, including **Vercel** (frontend hosting), **Render** (backend container execution), **MongoDB Atlas** (cloud database provision), **Cloudinary** (image management), and **Brevo** (automated SMTP email delivery).

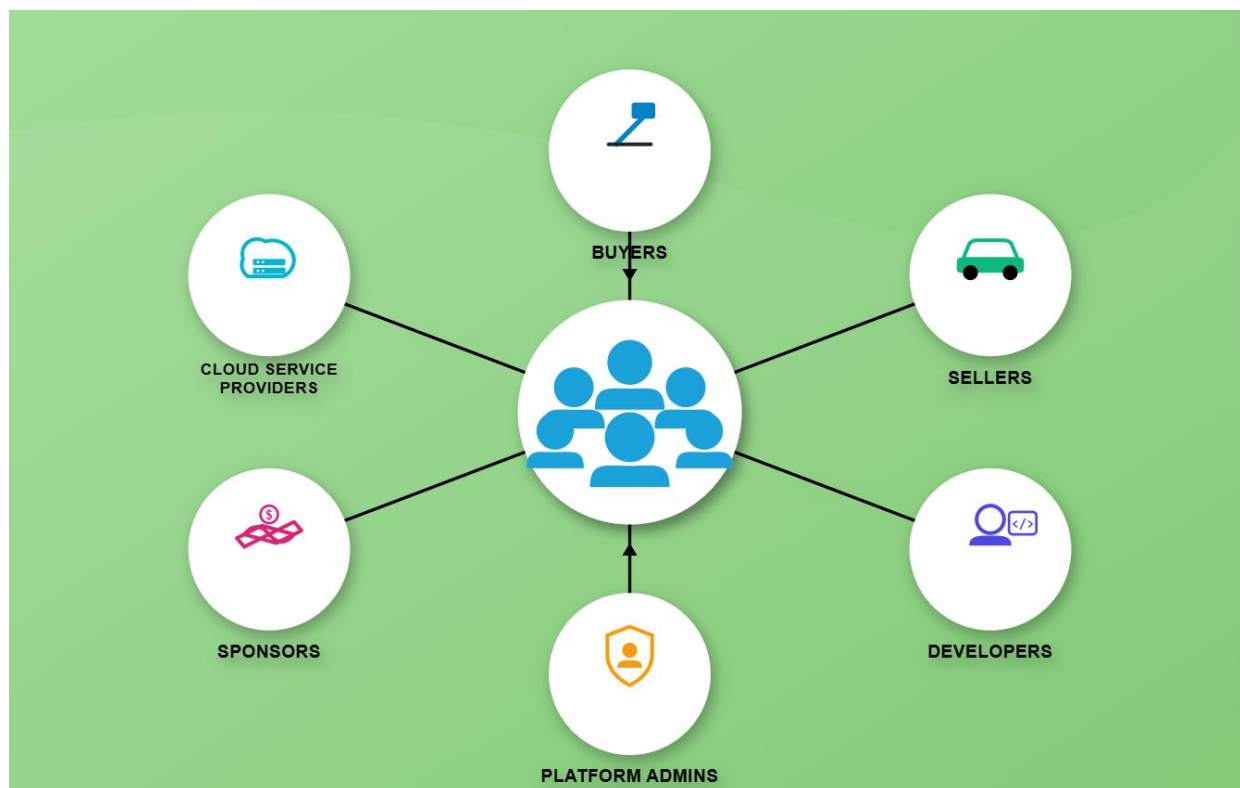


Figure 1 Stakeholders

Chapter 2 – MATERIALS & METHODS

2.1 Process Model:

The software development life cycle (SDLC) for the **Car Deposit** platform is engineered using a robust operational process model:

- **Agile Methodology:**

- **Iterative and Incremental Development:** The system is split into independent frontend and backend modules, allowing core structures like user registration and database schemas to be built first, followed by incremental feature updates like WebSocket bid streaming and CDN integration.
- **Regular Sprints:** Development is divided into 1-to-2-week structured sprints focused on adding specific functionalities, managing version control via GitHub, and performing automated verification routines.
- **Continuous Integration & Feedback:** Maintains a close feedback loop by evaluating performance benchmarks—such as ensuring real-time bid updates achieve a latency of under 500 milliseconds—to ensure maximum stability under heavy concurrent traffic.

- **Phases:**

1. **Requirement Analysis:** Identifying the crucial system components required for a premium auction engine, including JWT stateless tokens, Socket.io communication channels, and Cloudinary media uploading workflows.
2. **Design:** Structuring the split-stack architecture, creating indexed NoSQL collections in MongoDB Atlas, and establishing responsive multi-viewpoint mockups using Tailwind CSS.
3. **Development:** Implementing full-stack coding patterns utilizing NestJS (TypeScript) for the backend REST APIs/WebSockets and Next.js (React) for the client-side server-side rendered application layer.
4. **Testing:** Isolating service features for testing using the Jest framework, mocking external payment or notification gateways, and running end-to-end integration tests to completely eliminate concurrency race conditions.
5. **Deployment:** Running automated continuous delivery hooks to compile and push the live code instances directly to the Vercel (frontend) and Render (backend) cloud execution layers.
6. **Maintenance:** Monitoring server up-times, checking error logs for broken WebSocket connections, and planning future feature expansions such as Stripe payment gateways and AI recommendations.

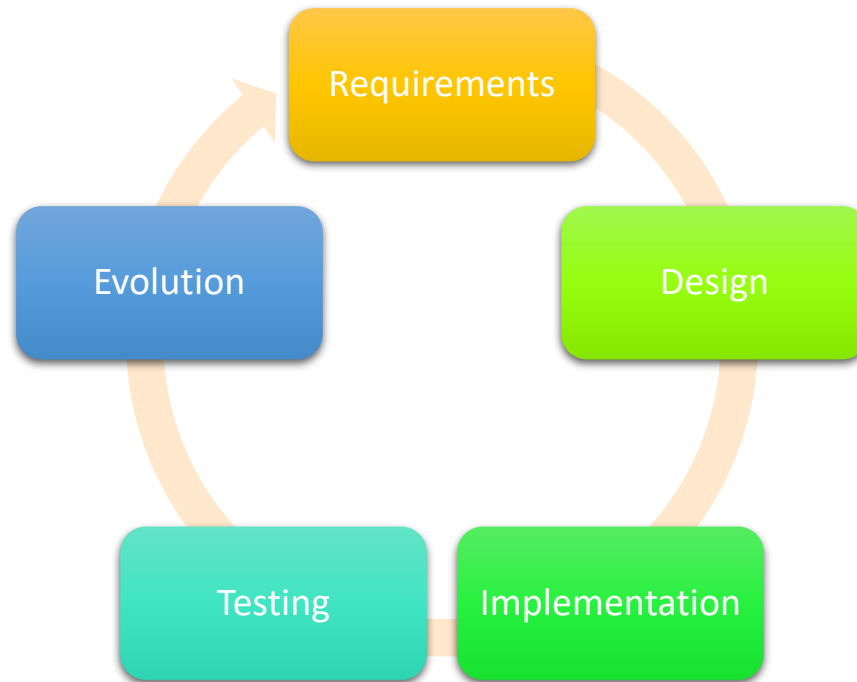


Figure 2 Agile Activities

2.2 Tools & Technologies

The precise engineering tools and technological framework utilized for building and maintaining the **Car Deposit** platform are detailed below:

- **Frontend Framework:** Next.js 14 (React 18) with TypeScript for building a fast, type-safe, and server-side rendered (SSR) user interface.
- **Styling Engine:** Tailwind CSS for creating a responsive, mobile-first visual layout across all screen sizes.
- **Backend Architecture:** NestJS (Node.js framework) built on top of Express to handle modular business logic and scalable REST APIs.
- **Real-Time Engine:** Socket.io (WebSockets) to enable bi-directional communication for instantaneous live bid updates and countdown timers.
- **Database Management:** MongoDB Atlas cloud database with Mongoose ODM for flexible, secure, and indexed document-based data storage.
- **Media Management:** Cloudinary SDK and CDN for secure, optimized direct uploads of high-resolution vehicle photography.
- **Notification System:** Brevo SMTP API integrated via Nodemailer for background transactional email alerts.
- **Testing and API Auditing:** Jest framework for isolating service unit tests, and Postman client for debugging API endpoints.
- **Version Control & CI/CD:** Git and GitHub for code synchronization, linked with automated GitHub Actions webhooks for continuous hosting deployment.

- **Cloud Hosting Platforms:** Vercel for serverless edge-network frontend hosting, and Render for containerized backend application execution.

2.3 Design:

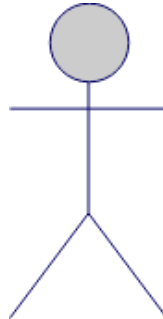
Note: Given below are some generic software design diagrams. However more design/UML content can be added according to the requirement and nature of your project.

2.3.1 Use Case Diagrams:

A Use Case Diagram is a high-level representation of the interactions between users (actors) and the system, describing all possible operations. The components in this use case diagram include:

Actors:

- **Buyer/Seller (User):** The primary user of the application who interacts with the system to register, log in via credentials or OAuth, list vehicles with images, place real-time bids, track active live auctions, and manage their personal wishlist.
- **Admin:** The system administrator responsible for monitoring backend platform health, managing user profile verifications, overseeing live marketplace auctions, and resolving active bidding disputes.



Actor

Use Cases:

- **Register:** New user creates a secure account on the marketplace.
- **Login:** Existing user logs in using standard credentials or secure OAuth.
- **Create Vehicle Listing:** Seller uploads vehicle details and high-resolution images to the cloud CDN.
- **Place Live Bid:** Buyer places instantaneous bids on active car auctions via Socket.io WebSockets.
- **View Auction Dashboard:** User monitors the real-time bid leaderboards and live countdown timers.
- **Manage Wishlist:** Buyer tracks specific vehicle listings for active bidding updates and instant alerts.

- **Track Auction History:** User reviews completed bid sequences and past transaction outcomes.
- **Generate Bidding Reports:** Admin or user generates summary sheets of auction metrics and wins.
- **Edit Profile:** User updates their personal dealership details, verification documents, or contact info.
- **Logout:** Securely terminates the session and clears authentication cookies.



Associations:

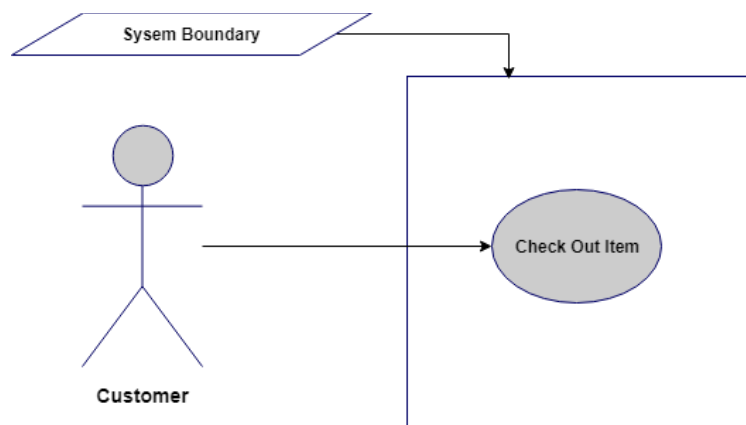
- **Buyer/Seller (User)** → All core application use cases (Register, Login, Create Vehicle Listing, Place Live Bid, View Dashboard, etc.)
- **Admin** → Associated with system operations, verification workflows, and the "Resolve Disputes" use case.
- **Representation** → Connections are shown with solid lines in the diagram.

System boundary:

Drawn as a rectangle that contains all car auction use cases.

Actors are placed outside this rectangle.

Title: Car Deposit

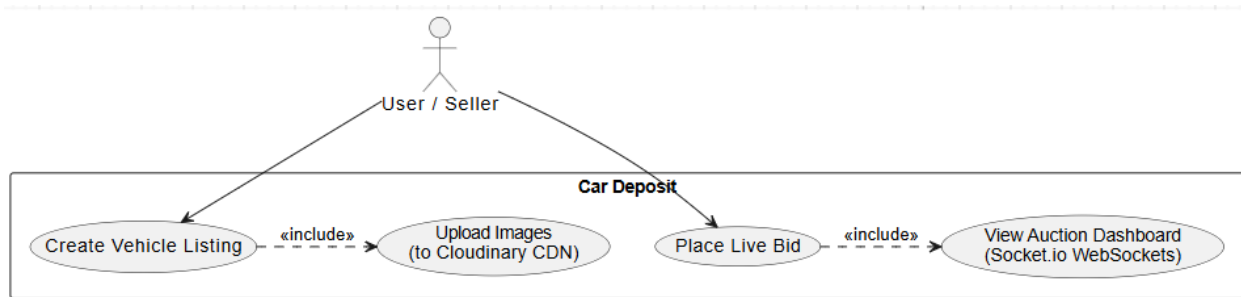


Relationship between Use cases:

- Include/Uses:**

Used when a base use case always calls or requires another use case to complete its process:

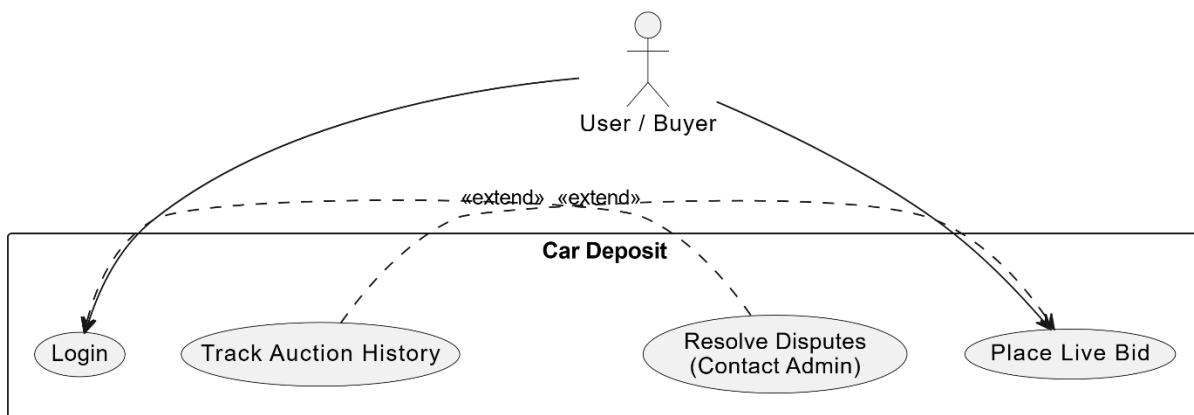
- **Place Live Bid** → <<include>> **View Auction Dashboard** (A user must be actively viewing the live dashboard and real-time countdown timer to place a valid bid.)
- **Create Vehicle Listing** → <<include>> **Upload Images** (A vehicle listing cannot be submitted or published without uploading its high-resolution media to the cloud CDN.)

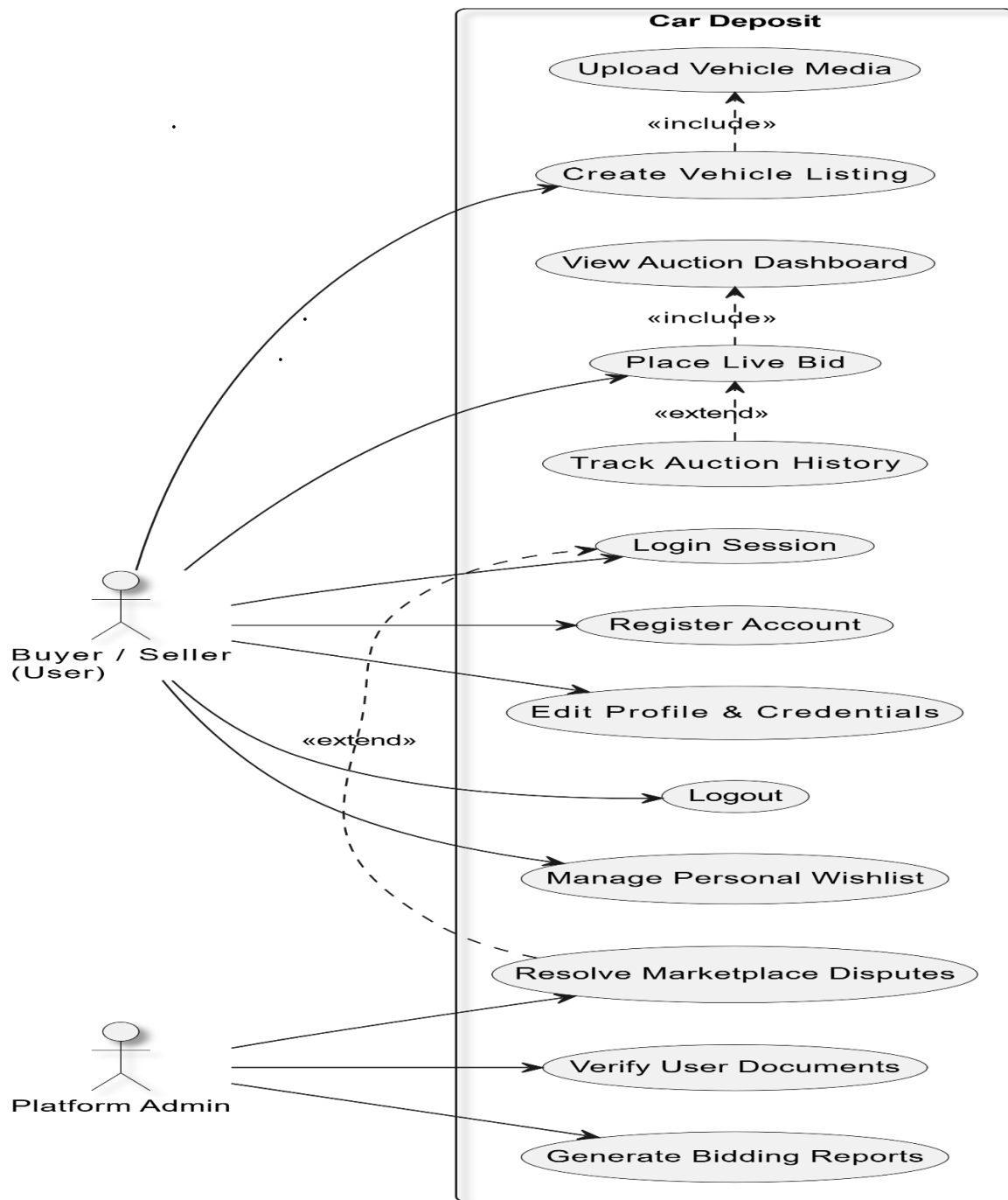


ii) Extend:

Used when a use case optionally extends the behavior of a base use case under specific conditions:

- **Login** → <<extend>> **Resolve Disputes (Contact Admin)** (Triggered only if the user encounters authentication errors, a locked account, or forgets their credentials—not during every login.)
- **Place Live Bid** → <<extend>> **Track Auction History** (If a buyer wins the auction or gets outbid, the system optionally redirects or updates their historical transaction logs.)



Use Case Diagram:*Figure 3 Use Case Diagram*

2.3.2 Usage Scenario

A Usage Scenario is a detailed, text-based explanation of a Use Case. It describes:

- How the buyer, seller, or admin interacts with the auction system step-by-step.
- What system conditions or states must be met before the interaction can begin.
- What transactional modifications occur after the process is completed.
- Any possible structural exceptions, and who has the access authority to execute it.

Each core use case mapped in your Use Case Diagram must have a separate standalone usage scenario.

Use Case Title	Write Title Here (must match use case title in use case diagram)	
Abbreviated Title		
Use Case Id		
Requirement Id		
Description:		
Pre Conditions:		
Task Sequence	Exceptions	
1.		
2.		
3.		
.		
.		
Post Conditions:		
Unresolved issues:		
Authority:		
Modification history:		
Author:		
Description:		

Let's explore each section from the template provided above. (you have to make only usage scenario tables, below is description of its each part)

▪ Use Case Title

- This is the specific name of the use case under description. It always begins with an action verb (like "Upload", "View", "Generate").
- It must match exactly with the use case in your diagram.
- Example:

Use Case Title: Create Vehicle Listing

Use Case Title	Create Vehicle Listing
-----------------------	-------------------------------

▪ **Use Case Id**

- A unique identifier for this use case.
- Helps in referencing and managing documentation.
- Example:
Use Case Id: 3

▪ **Requirement Id**

- Refers to the Functional Requirement ID from your system requirements (FR01, FR02, etc.).
- Shows which system requirement this use case fulfills.
- Example:
Requirement Id: FR03

Requirement Id	3
-----------------------	---

▪ **Description**

- A brief summary (2–3 lines) of what the use case does.
- What is the purpose of this action?
- What outcome is expected?
- Example:
“This use case enables an authenticated Seller to submit a comprehensive vehicle portfolio including structural specifications and media assets, which are then processed by the platform engine to initialize a live public auction entry.”

Description	This use case enables an authenticated Seller to submit a comprehensive vehicle portfolio including structural specifications and media assets, which are then processed by the platform engine to initialize a live public auction entry.
--------------------	--

▪ **Pre-Conditions**

- Conditions that must be **true before** the use case can begin.
- These can be technical, user-related, or environmental.
- **Examples:**
 - The user must be logged in.

- The device must have a camera.
- Internet must be connected.

Pre Conditions:

1. The application server must confirm a valid, unexpired JWT stateless session token for the user.
2. Connection to the remote database cluster and object storage CDN must be active and operational.

▪ **Task Sequence & Exceptions**

- A step-by-step breakdown of what the user does and how the system responds.
- Describes the entire interaction from start to finish.
- Example Steps:
 1. User taps on "Add Vehicle Media".
 2. System launches device gallery or file manager.
 3. User selects high-resolution vehicle photographs.
 4. System processes, optimizes, and previews the selected images.

Task Sequence	Exceptions
1. User selects the option to launch a new live car auction listing.	
2. System requests vehicle details, bidding conditions, and multimedia assets.	
3. User supplies all mandatory information and confirms the submission request.	
4. System verifies parameters and updates the real-time marketplace database.	
5. System broadcasts live auction telemetry alerts and dispatches confirmation emails.	
6. A permanent audit log is registered upon successful completion of the sequence.	

▪ **Post Conditions**

- What is the final result after the use case is completed successfully?
- Example:

“A new live vehicle auction listing is successfully created and broadcasted.”

Post Conditions:

- A new live vehicle auction listing is successfully created and broadcasted.

- **Unresolved Issues**

- Anything pending or unclear that needs to be addressed later.
- If nothing is unresolved, just write: None

- **Authority**

- Which user role is allowed to perform this use case?
- Example:
Authority: Buyer / Seller (User)

- **Modification History**

- Tracks any changes made to the use case scenario over time.
- For your first version, just write:
Modification History: Version 1.0

- **Author**

- The team, group, or individual who wrote the use case.
- Example:
Author: CarDeposit_2026

- **Description**

- Any additional details or notes.
- You can leave it blank if not needed

Table 2. 1: Register Account

Use Case Title	Register Account
Use Case Id	UC_01
Requirement Id	FR_01
Description: This use case is about adding a new user (Buyer/Seller) to the existing system with the privileges defined at the time of user account creation.	
Pre Conditions:	
1. All must-required information about the new user should be available.	
2. Database should be available in online mode.	
Task Sequence	Exceptions

1. User opts to register a new user account.	
2. System asks for necessary information.	
3. User provides all the required information and opts to complete the operation.	
4. There is a problem in the data provided; some data needs to be corrected. – User checks the available information and corrects the error. – User continues from step 3.	
5. System after confirmation adds the new account.	
6. System sends the account creation email to the administrator's email id and user's email address.	
7. A log is saved on the successful operation of the use case.	
Post Conditions: – A new user account is successfully created.	
Unresolved issues:	
Authority: Administrator	
Modification history: 1.0	
Author: <Project or Group ID>	
Description:	

Important Points:

- ✓ Each use case (e.g., Register, Login, Generate Report) will have a separate scenario like this.
- ✓ Titles must match exactly with the Use Case Diagram.
- ✓ If there's nothing to write under “Unresolved Issues” or “Modification History,” just write None or Version 1.0.
- ✓ Mention "User" or "Admin" under "Authority" based on which actor uses the case.

2.3.3 Sequence Diagram:

A Sequence Diagram is a type of UML (Unified Modeling Language) diagram that shows how different parts of a system interact with each other over time.

It focuses on the order of operations, showing who initiates what, and how messages are passed between objects or components.

Sequence Diagram Notations:

Each part of the diagram has a specific purpose. Below are detailed explanations of the main components:

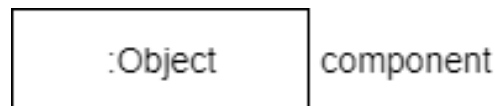
Actors:

- Actors are external users or systems that interact with your application.
- They are always outside the system boundary.
- In your Car Deposit Platform, typical actors include:

- **User (Buyer/Seller):** The person using the app to create vehicle listings, place live bids, view dashboard metrics, etc.
- **Admin:** An optional actor who helps with password recovery.

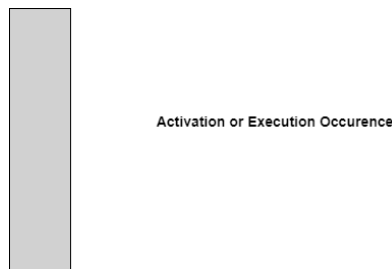
Class Roles or Participants:

- These are the internal components or objects of the system that participate in the interaction.
- Examples: Controller, API Handler, Database, WebSocket Server.
- They are represented as boxes at the top of the diagram, each with a vertical line called a lifeline.
- In your app, typical participants might be:
 - **AuthenticationService** (Handles user login, registration, and JWT/OAuth generation)
 - **VehicleListingController** (Manages automobile details and communicates with Cloudinary CDN)
 - **LiveAuctionWebSocket** (Handles instantaneous, real-time bid broadcasts using Socket.io)
 - **BiddingReportGenerator** (Compiles historical auction metrics and transactions into downloadable reports)



Activation or Execution Occurrence:

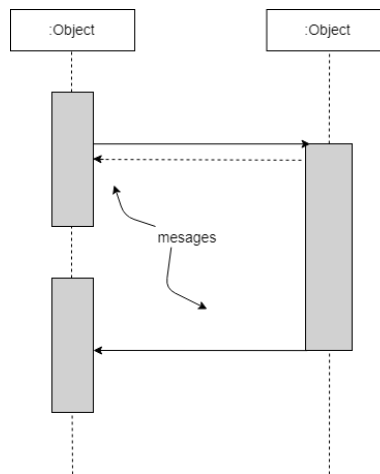
- This represents the duration in which an object is performing a specific operation.
- It's shown as a thin vertical rectangle on the object's lifeline.
- When a method or function is called, the rectangle appears on the lifeline.
- The longer the rectangle, the longer that component is "active" or "processing".



Messages

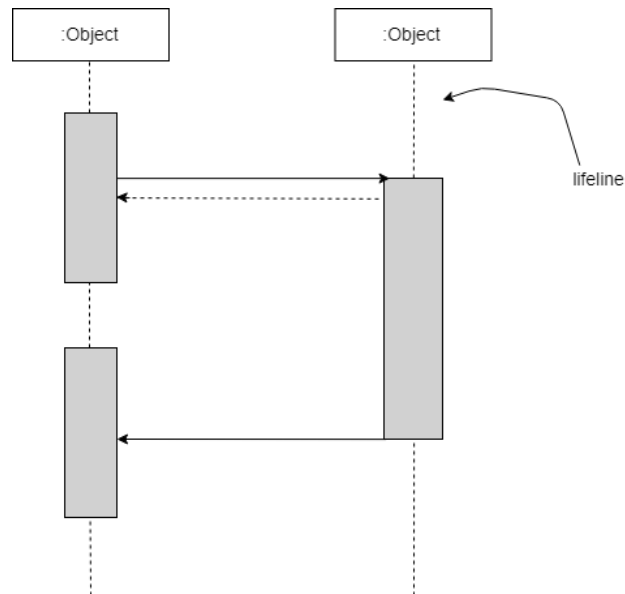
- Messages are the arrows that show how objects communicate.

- These can be method calls, data being passed, or responses.
- There are two types:
 - Synchronous Message: The sender waits for the response (solid arrow with filled arrowhead).
 - Asynchronous Message: The sender doesn't wait (solid arrow with open arrowhead).
- Example in your app:
 - User → App: “Create Vehicle Listing”
 - App → Model: “Broadcast Live Bid”



Lifelines

- A lifeline is a dashed vertical line below each participant box.
- It represents the object's existence over time.
- Every participant in the interaction has its own lifeline.
- When an object is active (executing), the activation box appears on the lifeline.



Destroying Objects

- This represents termination of an object after it's no longer needed.
- It's shown with an "X" at the end of the lifeline.
- For example, if a temporary object like a session or upload handler is destroyed after completing its job, you mark its destruction at the end of the interaction.

Loops

- Loops are used to show repetitive actions.
- They are drawn as a box around multiple messages.
- A condition is mentioned like: [for each image] or [while image count < 4]
- Useful when you're showing something like:
 - User uploads 4 images → system processes them one by one.

Sequence Diagram Example (for item return use case):

Note: Make sequence diagram for each use case illustrated in use case diagram

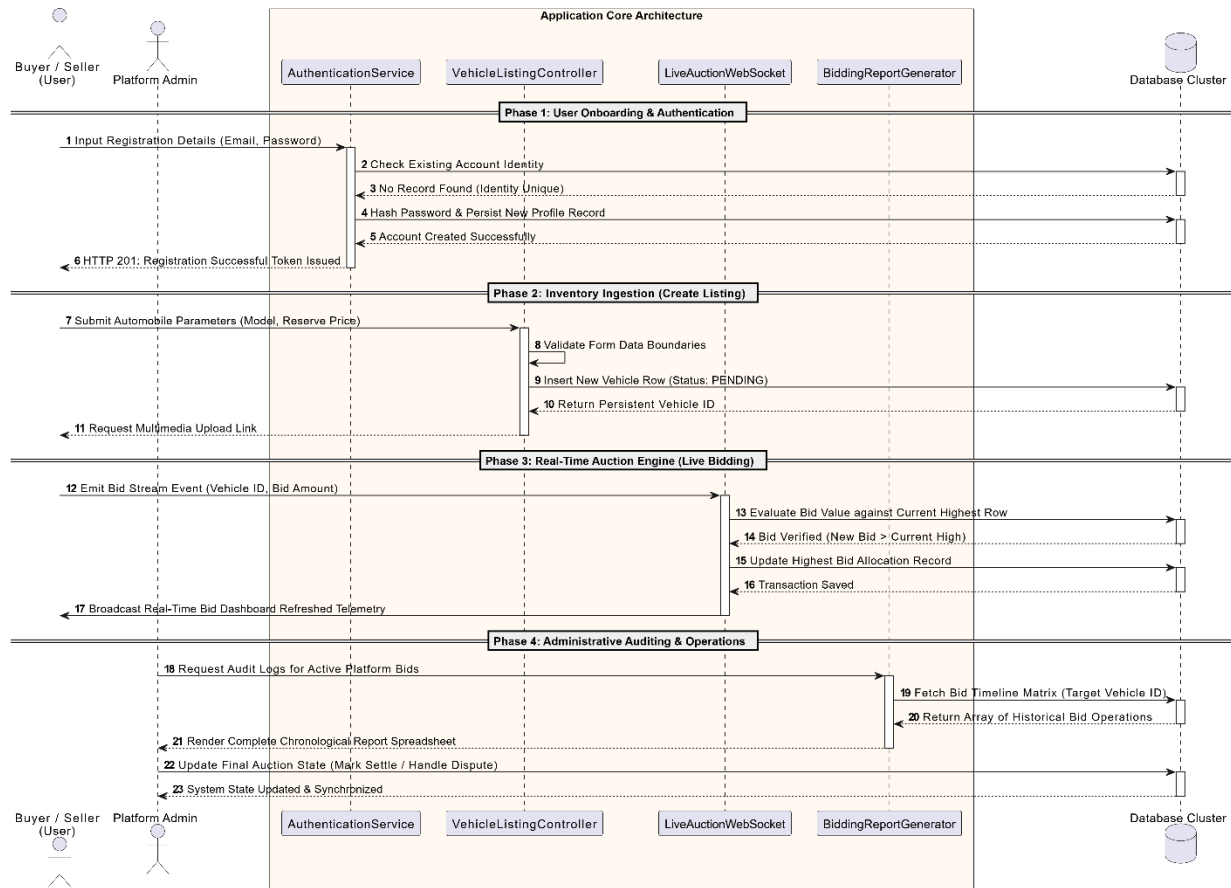


Figure 4 Sequence Diagram

2.3.4 Class Diagram:

A Class Diagram represents the structure of a system by showing its classes, their attributes, and methods. It helps define how different parts of the application are organized and how they interact. In **Car Deposit Platform**, important classes include `User`, `VehicleListing`, `Bid`, and `BiddingReport`. Relationships like *"User creates VehicleListing"* and *"User places Bid"* show how the system components connect logically.

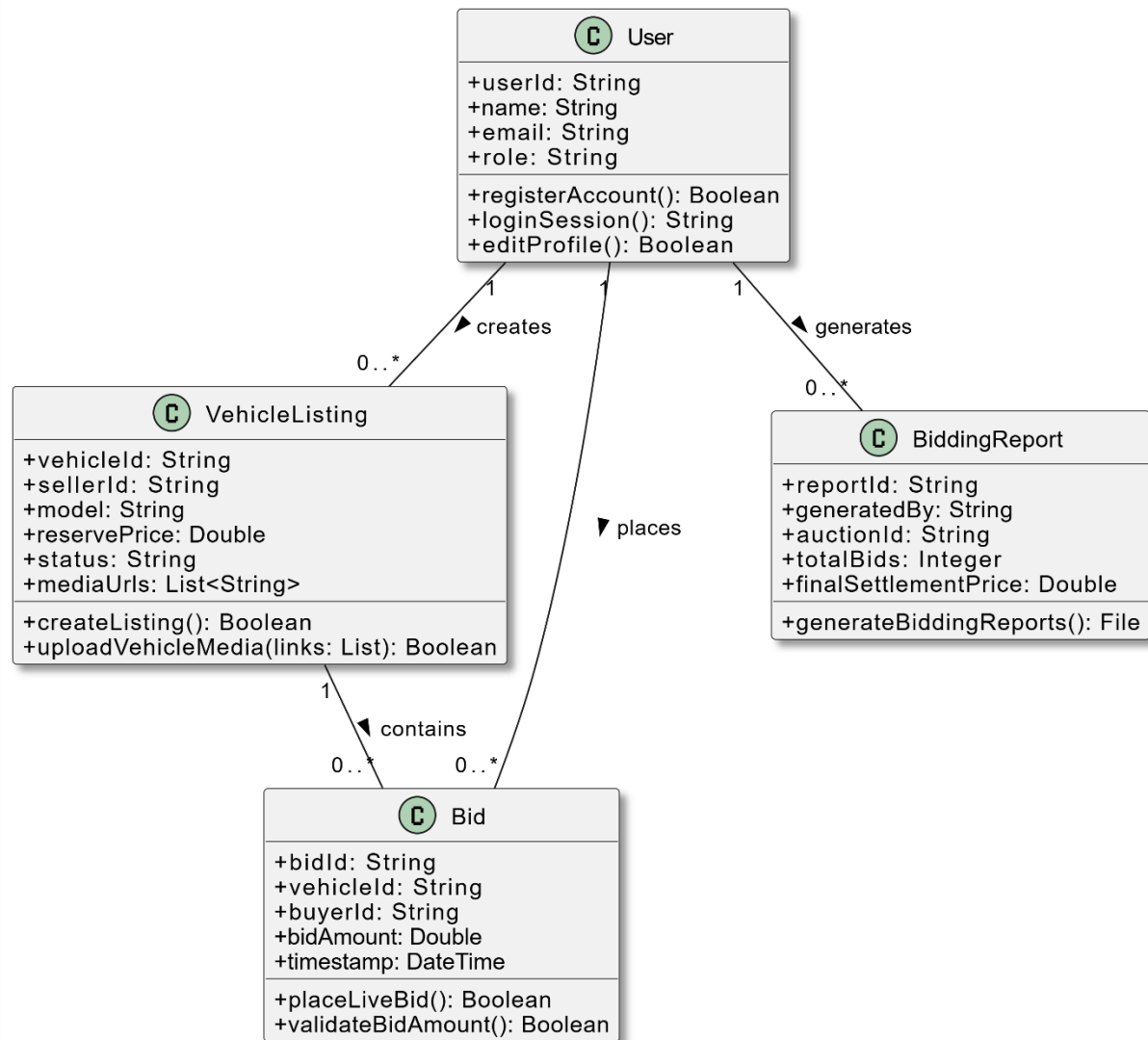


Figure 5 Class Diagram

2.3.5 Data Flow Diagram:

A Data Flow Diagram (DFD) is a graphical tool used to represent the flow of data through a system. It shows how data is input, processed, stored, and output, without detailing the programming logic.

DFDs consist of four main components:

1. Processes – Circles or ovals that represent actions/functions (e.g., “Upload Image”, “Count Citrus”).
2. Data Stores – Repositories like databases where data is saved (e.g., “User Data”, “Image Records”).

3. External Entities – Outside systems or users (e.g., Farmer, Admin) that interact with the system.
4. Data Flows – Arrows that show how data moves between processes, entities, and stores.

The DFD is typically shown in levels:

- Context Diagram (Level 0): High-level overview showing only one process and external entities.
- Level 1 DFD: Breaks down the main process into sub-processes like registration, image upload, and report generation.
- Level 2 DFD (optional): Provides even more detail of individual steps.

This diagram helps developers, stakeholders, and testers understand how data moves through the system, ensuring the app is logically designed and complete.

DFD Diagram Notations

External Entity

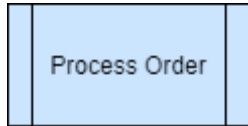
- Represents any outside person, system, or organization that interacts with your application.
- It either sends data into the system or receives data from it.
- Drawn as a rectangle.
- In your **Car Deposit Platform**, examples include:
 - **User / Buyer / Seller** (creates vehicle listings, uploads media, places live bids, views auction dashboards)
 - **Admin** (verifies user documents, resolves marketplace disputes, generates bidding reports)



Process

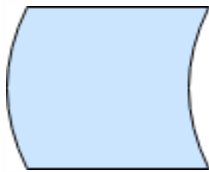
- Represents an action or operation that transforms input data into output.
- Drawn as a circle or rounded rectangle.
- Examples from app:
 - “Register User Account”

- “Create Vehicle Listing”
- “Process Live Bid”
- “Generate Bidding Report”



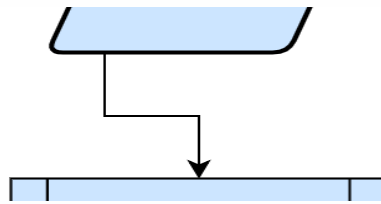
Data Store

- Represents a place where data is stored within the system.
- Drawn as two horizontal lines (or sometimes an open-ended rectangle).
- Examples:
 - User Database
 - Image Repository
 - Report History



Data Flow

- Shows the movement of data between entities, processes, and data stores.
- Drawn as a labeled arrow.
- Example flows:
 - **Vehicle details & images** → from User to "Create Vehicle Listing" process
 - **Live bid amount** → from Buyer to "Process Live Bid" process



Context Diagram:

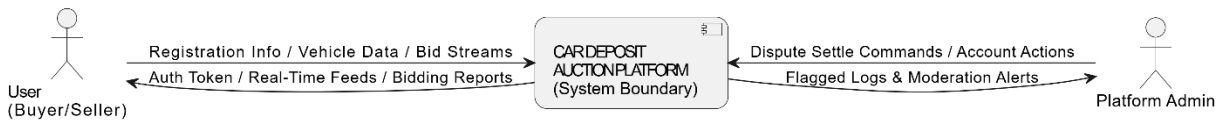


Figure 6 Context Diagram

Level 0:

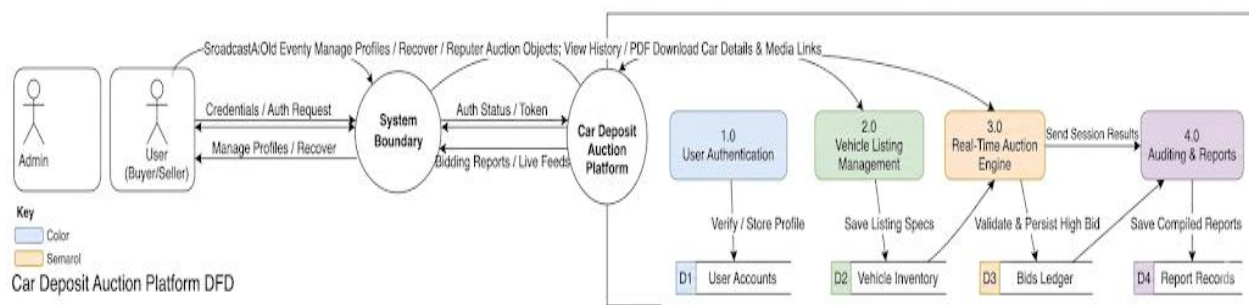


Figure 7 Level 0 DFD

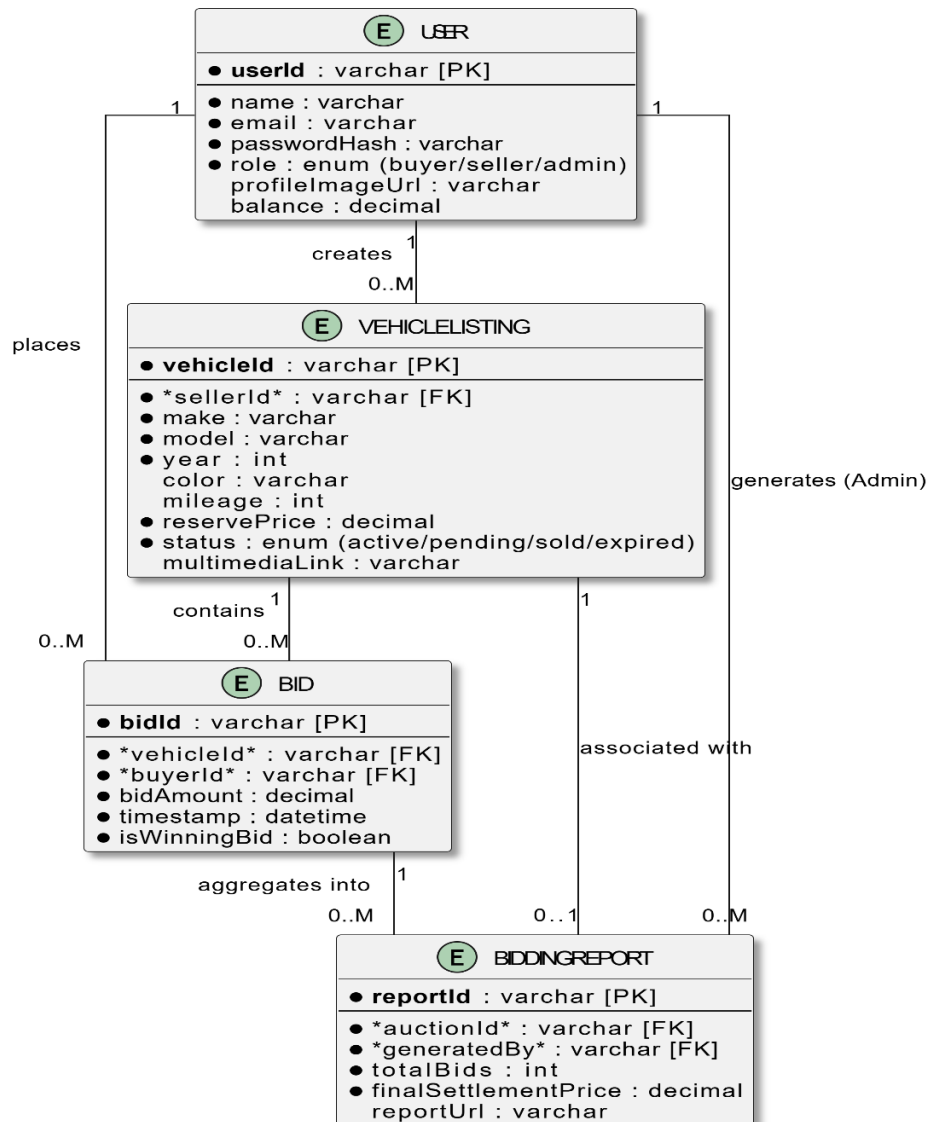


Figure 9 Entity Relationship Diagram

2.3.7 Database Model:

A Database Model defines the structure and organization of the database used by the system. It describes:

- Tables (or entities) that store data
- Columns (attributes) inside each table
- Primary keys to uniquely identify records
- Foreign keys that link tables together
- Relationships between tables (one-to-one, one-to-many)

This model helps developers understand how data is stored and accessed efficiently.

Database Model for Car Deposit Platform

The main tables in the database are:

- **User:** Stores user details and authentication metadata (`user_id`, name, email, password_hash, role, balance).
- **Vehicle_Listing:** Stores structural details of automobiles uploaded for bidding (`vehicle_id`, model, year, reserve_price, status, `user_id` FK).
- **Bid:** Stores real-time chronological bid entries received during live streams (`bid_id`, bid_amount, timestamp, is_winning_bid, `vehicle_id` FK, `user_id` FK).
- **Bidding_Report:** Stores compiled audit logs and final settlement sheets generated by admins (`report_id`, total_bids, final_settlement_price, report_url, `vehicle_id` FK, `user_id` FK).

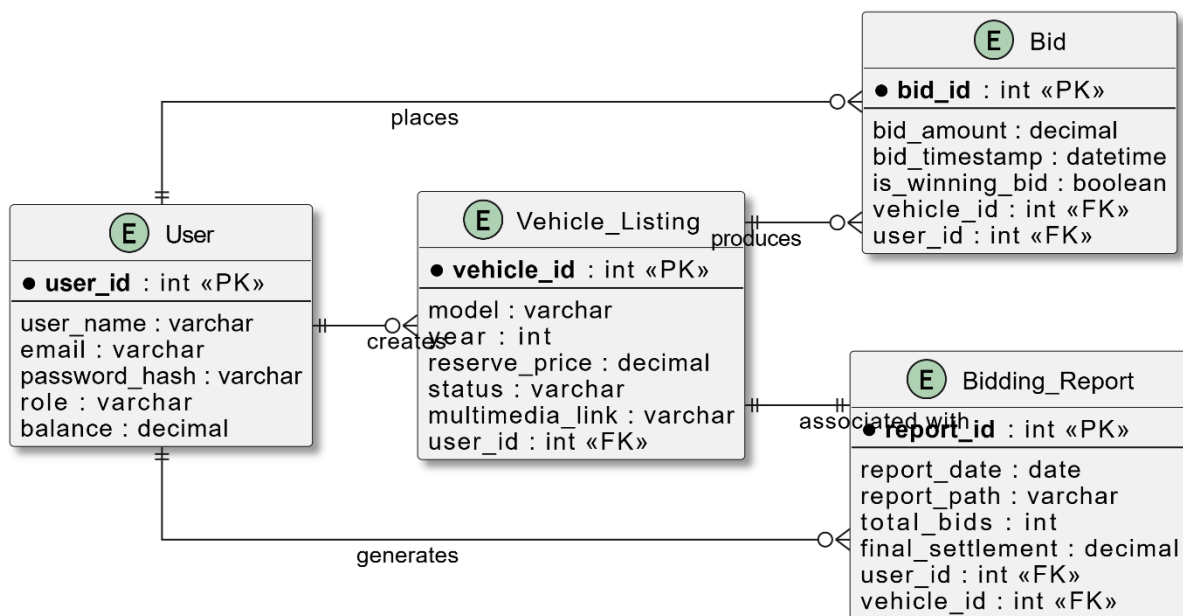


Figure 10 Database Model

2.3.8 Architecture:

The Car Deposit Platform follows a three-tier architecture, which separates the application into three main layers:

1. Presentation Layer (Client Layer)

- This is the user interface where buyers, sellers, and administrators interact with the platform.
- It includes the web dashboard screens for user onboarding, car specification submission, media galleries, administrative conflict logs, and the real-time live bidding matrix.
- This layer captures user inputs (such as registration forms and active bid event keystrokes) and renders real-time processed telemetry data.
- It establishes standard stateless HTTP channels for asset setup and maintains persistent client-side WebSocket links to stream instant auction room price changes.

2. Application Layer (Business Logic Layer)

- This layer contains the core processing logic and operational rules of the platform.
- It intercepts incoming traffic modules from the interface, executes strict input boundary validation, and coordinates chronological workflows.
- It manages user identity checking (via password hashing and JWT token issuance), structural inventory publishing validations, and processes backend data calculations to compile final auction audit sheets.
- It hosts the event-driven WebSocket gateway engine that continuously evaluates live bid amounts, manages concurrent bidding streams, and instantly broadcasts validated highest bid entries back to the client pool.
- This layer acts as the foundational processing bridge between user actions and the data layer.

3. Data Layer (Database Layer)

- This layer manages the secure storage, relational binding, and transactional retrieval of system records.
- It includes highly structured database clusters that store persistent user account entities, item configurations, chronological bids tables, and administrative logging arrays.
- The application layer communicates with this layer using query operations or an object-relational mapping interface (ORM) to retrieve profile validation states, write active records, or run heavy analytical fetches to compile transaction spreadsheets.

Why Three-Tier Architecture?

- **Modularity:** Each layer is independent, making the system easier to maintain and update.
- **Scalability:** Layers can be scaled separately to handle more users or data.
- **Security:** Sensitive data is managed at the data layer, separated from the user interface.

- Reusability: Business logic can be reused for different clients (mobile app, web app).

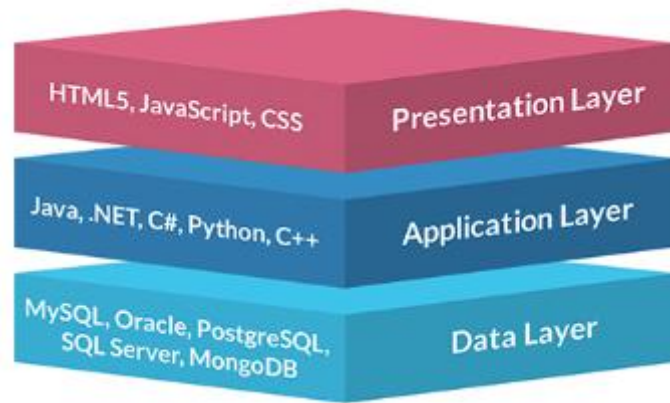


Figure 11 Application Architecture

Chapter 3 - RESULTS & DISCUSSION

In this chapter, the overall performance and all functional and non-functional requirements are discussed to verify the performance measures proposed for this project. For this, software testing plays a vital role.

3.1 Testing:

Software testing is a crucial process to evaluate whether the Car Deposit Platform meets the functional and non-functional requirements defined in Chapter 1. The aim of testing is to ensure that the application performs reliably under high concurrency, delivers accurate transactional results, and provides a secure, user-friendly experience for buyers, sellers, and administrators. This includes checking the real-time latency of the live bidding engine, the correctness of asset data submission, cryptographic token management, and payment/balance ledger validation.

Test case writing played a key role in the quality assurance of this project. Test cases were designed to cover all major features including the Live Bidding Dashboard, Vehicle Inventory Upload, Real-Time WebSocket Streaming, User Onboarding & Authentication, Admin Moderation Panels, and Automated Audit Report Generation modules. Each test case clearly defined the technical steps, expected outputs, and actual outcomes to verify system stability. These test cases serve as a reference for the development and testing teams, and validate that the Car Deposit Platform functions seamlessly as intended.

3.2 Test Cases:

3.2.1 User Login:

Test Case ID:	TC-1
Test Case Title:	To Verify registered User can log in successfully with valid credentials
Test Case Priority:	High
Requirement:	Authentication and Secure Access
Test Description:	This test verifies that an existing user can log into the platform using correct login details and get an authorized session.
Test Date:	18/05/2026
Pre-Conditions:	1. User is registered in the database. 2. User is on the Login screen.
Dependencies:	Authentication Service, Database Connection
Test Steps:	1. Enter registered Email and valid Password. 2. Click on the "Login" button.

	3. Verify the server dashboard redirection response
Test Data	email: "mehak@example.com", password: "SecurePass123!"
Expected Results:	1. Credentials are verified against the password hash in the database. 2. App receives a valid JWT token.
Actual Results:	As expected
Post Conditions:	User state becomes active; session token is stored in client storage.
Status: (Pass/Fail)	Pass
Other Comments:	None

3.2.2 User Login with Invalid Credentials:

Test Case ID:	TC-2
Test Case Title:	To Verify Login failure and error messages with wrong credentials
Test Case Priority:	High
Requirement:	Security and Error Handling
Test Description:	This test ensures that the system rejects access when incorrect email/password combinations are provided.
Test Date:	18/05/2026
Pre-Conditions:	User is on the Login screen.
Dependencies:	Authentication Service
Test Steps:	1. Enter an unregistered email or an incorrect password. 2. Click the "Login" button. 3. Observe input field highlighting and toast notifications.
Test Data	email: "wronguser@example.com", password: "WrongPass!"
Expected Results:	1. Authorization fails at the backend layer. 2. App displays an error notification stating "Invalid Email or Password". 3. User remains on the login screen; no session token is generated.
Actual Results:	As Expected
Post Conditions:	System blocks unauthorized access attempts.
Status: (Pass/Fail)	Pass
Other Comments:	Form layout fields clear the password box automatically after failure.

3.2.3 Forget Password Screen:

Test Case ID:	TC-3
Test Case Title:	To verify "Forgot Password" shows admin contact information
Test Case Priority:	Low
Requirement:	User should be able to contact admin if they forget password
Test Description:	This test checks whether the user can access the admin contact number and how they can contact the admin to retrieve a new password.
Test Date:	18/05/2026
Pre-Conditions:	1. App is installed and user is on Login Screen. 2. Admin contact information is available.
Dependencies:	Internet Availability and "Forgot Password" feature is working and visible
Test Steps:	1. Tap on "Forgot Password". 2. A new screen appears with the admin's Whatsapp number. 3. Enter your name and phone number. 4. Tap the "Send" button. 5. Admin receives the request and provides the new password.
Test Data	1. User's name: Bisma 2. Phone Number: 03XXXXXXXXX
Expected Results:	1. Admin's phone number should appear 2. After entering user details and tapping "Send," the admin should receive the request and provide the new password
Actual Results:	Admin number displayed correctly. WhatsApp opened with pre-filled message and sent successfully.
Post Conditions:	User has received the new password from the admin.
Status: (Pass/Fail)	Pass
Other Comments:	Admin's contact information displayed successfully. User was able to send their details, and the admin provided a new password

3.2.4 Signup Screen:

Test Case ID:	TC-4
Test Case Title:	To verify the Signup functionality of the application
Test Case Priority:	High
Requirement:	User should be able to sign up with valid credentials. Proper error messages must be displayed for invalid inputs.
Test Description:	This test verifies that the Sign Up screen accepts only valid inputs and shows correct validation messages for invalid ones..
Test Date:	18/05/2026
Pre-Conditions:	1. App is installed and running. 2. User is on the Sign Up screen.
Dependencies:	Working backend and internet connectivity
Test Steps:	1. Open app. 2. Enter valid full name, phone number (03XXXXXXXXX), password (6+ characters), and matching confirm password and tap Signup.

	3. Leave full name empty and tap Sign Up. 4. Enter invalid phone number format and tap Signup. 5. Leave phone number field empty and tap Signup. 6. Enter password less than 6 characters and tap Signup. 7. Leave password field empty and tap Signup. 8. Enter non-matching confirm password and tap Signup. 9. Leave all fields empty and tap Signup.
Test Data	Full Name: Bisma Phone Number:03XXXXXXXXXX Password:abc@12 Confirm Password:abc@12
Expected Results:	1. User is successfully signed up and redirected to home screen. 2. Error shown: "Full Name is required" 3. Error shown: "Enter a valid phone number" 4. Error shown: "Phone Number is required" 5. Error shown: "Password must be at least 6 characters" 6. Error shown: "Password is required" 7. Error shown: "Passwords do not match" 8. Error shown: "All fields are required"
Actual Results:	Signed up correctly and navigated to Home screen.
Post Conditions:	1. On successful sign up, user is navigated to the home screen. 2. On invalid input, user stays on Sign Up screen and receives relevant validation messages.
Status: (Pass/Fail)	Pass
Other Comments:	All field validations, error messages, and navigation are functioning as expected.

3.2.5 View Vehicle Auction Room History:

Test Case ID:	TC-5
Test Case Title:	To Verify User can fetch and view past historical lists rows of auctions
Test Case Priority:	Medium
Requirement:	Inactive Record Tracking
Test Description:	Verifies that buyers and sellers can view logs of concluded or past bidding history rooms.
Test Date:	18/05/2026
Pre-Conditions:	User is logged in and has navigated to the "Auction History" dashboard section..
Dependencies:	Report Records, Vehicle Inventory Data Store
Test Steps:	1. Access the history panel dashboard view. 2. Filter by date or vehicle type if required.

	3. Observe item cards rendering.
Test Data	User ID token data reference mapping
Expected Results:	1. Backend pulls records where status is sold or expired from the data layer. 2. App renders historical rows detailing the final winning bid price and settlement stats accurately.
Actual Results:	As expected
Post Conditions:	None
Status: (Pass/Fail)	Pass
Other Comments:	Inactive lists load quickly due to index optimization parameters.

3.2.6 Images Screen:

Test Case ID:	TC-6
Test Case Title:	To verify image uploading functionality gallery options and validation for minimum 4 images
Test Case Priority:	High
Requirement:	The user should be able to upload exactly images Select from Gallery, and only then the Proceed button should enable and navigate to the Summary page
Test Description:	This test ensures the user can upload 4 images and cannot proceed without doing so. It verifies the camera and gallery functionalities and checks the transition to the Summary screen.
Test Date:	18/05/2026
Pre-Conditions:	1. User is logged in and navigated to the Image Upload screen. 2. Camera and storage permissions are granted
Dependencies:	1. Camera and Gallery permissions granted 2. Summary page is linked correctly 3. Button state must be dependent on image count
Test Steps:	1. Open the app and navigate to the Image Upload screen 2. Tap on the first "Add Image" button 3. Tap on "Capture Image" or "Select from Gallery". 4. Repeat these steps for the remaining slots. 5. After adding 4 images tap the "Proceed" button.
Test Data	4 different images
Expected Results:	1. Add Image" triggers the modal correctly 2. Both image sources (camera/gallery) function as expected 3. "Proceed" button is disabled until 4 images are uploaded 4. After 4 images, "Proceed" becomes clickable 5. Clicking "Proceed" navigates to the Summary screen

Actual Results:	1. Image options (Capture/Gallery) opened correctly on clicking "Add Image". 2. Images displayed in slots after selection. 3. "Proceed" button stayed disabled until 4 images were added. 4. After 4 images, "Proceed" enabled and navigated to Summary screen.
Post Conditions:	User reaches the Summary screen with 4 uploaded images.
Status: (Pass/Fail)	Pass
Other Comments:	Image upload and navigation functioned correctly with responsive UI.

3.2.7 Summary Screen:

Test Case ID:	TC-7
Test Case Title:	To verify citrus count display, previous history, and navigation on Citrus Summary screen
Test Case Priority:	High
Requirement:	The user should see the total citrus count from uploaded images, view previous counts with dates, and be able to generate a report or exit the app
Test Description:	This test ensures that the citrus count from backend processing appears correctly, previous history is visible, and navigation buttons work as intended..
Test Date:	13/05/2025
Pre-Conditions:	1. User has uploaded 4 images and reached the Citrus Summary screen. 2. Backend model is properly trained and integrated. 3. Internet connection is active.
Dependencies:	1. Backend processing for citrus detection must be functional. 2. Previous history data should be available in the database.
Test Steps:	1. Navigate to Citrus Summary screen after uploading 4 images. 2. Observe the "Total Citrus Count" section. 3. Check if the count is shown after loading completes. 4. Scroll down to view "Previous History". 5. Tap on the "Generate Report" button. 6. Tap on the "Exit App" button.
Test Data	Citrus Images uploaded
Expected Results:	1. Citrus count appears after loading completes. 2. Previous history list displays with count and dates. 3. "Generate Report" opens Report screen. 4. "Exit App" closes the application.
Actual Results:	1. Total Citrus Count showed "Loading..." initially, then displayed correctly. 2. History section showed no previous data (first-time use). 3. Generate Report navigated to the Report screen. 4. Exit App closed the application.

Post Conditions:	User views citrus count, navigates to report or exits as needed.
Status: (Pass/Fail)	Pass
Other Comments:	Citrus count displayed correctly. History not visible due to first time use. Navigation works fine.

3.2.8 Profile Screen:

Test Case ID:	TC-8
Test Case Title:	To verify the profile screen's functionality including contact info display, edit profile navigation, and logout confirmation.
Test Case Priority:	High
Requirement:	The user should be able to view their profile information, access the Edit Profile screen, and confirm logout upon pressing the logout button.
Test Description:	This test ensures the profile screen components are correctly displayed and functional, including data fetched from signup, screen navigation, and logout confirmation dialog.
Test Date:	18/05/2026
Pre-Conditions:	1. User is logged in and navigates to the Profile screen. 2. User's name and contact info are stored from the Signup process.
Dependencies:	1. User info fetched from Signup screen. 2. Edit Profile screen must be implemented. 3. Logout confirmation dialog must be implemented
Test Steps:	1. Launch the application and navigate to the Profile screen. 2. Observe the display of profile image/icon, "My Profile" heading, and user name. 3. Tap on the "My Contact Info" section. 4. Verify that the correct phone number (from signup) is displayed. 5. Tap on "Edit Profile". 6. Confirm navigation to a new screen titled "Edit Profile". 7. Tap on the "Log Out" button. 8. Verify that a dialog appears saying: "Are you sure you want to log out?"
Test Data	User Name: Bisma Phone Number: 03XXXXXXXXXX
Expected Results:	1. My Profile" and user name are displayed. 2. Contact number fetched from signup appears correctly on tap. 3. Tapping "Edit Profile" opens a screen titled "Edit Profile". 4. Logout button opens a confirmation dialog: "Are you sure you want to log out?"
Actual Results:	1. Profile info displayed correctly. 2. Contact number shown accurately. 3. Edit Profile screen loaded on tap. 4. Logout confirmation appeared as expected.

Post Conditions:	User can update profile info, view contact, or proceed with logout after confirmation.
Status: (Pass/Fail)	Pass
Other Comments:	All functionalities worked as intended. Smooth navigation and clear UI presentation.

3.2.10 Edit-Profile Screen:

Test Case ID:	TC-10
Test Case Title:	To verify that user details can be updated and saved to the backend through the Edit Profile screen
Test Case Priority:	High
Requirement:	The user must be able to update their full name, phone number, and password, and save the data to the backend by clicking the "Save" button.
Test Description:	This test ensures that all fields on the Edit Profile screen accept input correctly, and upon clicking the "Save" button, the updated data is successfully sent to and stored in the backend.
Test Date:	13/05/2025
Pre-Conditions:	1,. User is logged in and navigates to the Edit Profile screen. 2. User already has existing profile data from Signup. 3. Backend endpoint for updating user data is available and functional.
Dependencies:	Backend API, active internet, and form validation must be in place.
Test Steps:	1. Open the app and navigate to the Edit Profile screen. 2. Tap on the Full Name field and enter a new name. 3. Tap on the Phone Number field and update the number. 4. Tap on the Password field and enter a new password. 5. Tap the green Save button. 6. Observe success message or profile update.
Test Data	Full Name: Ali Phone Number: 03XXXXXXXXX Password: Ali@12
Expected Results:	1. All input fields accept valid data. 2. Save button triggers a backend API call. 3. User sees a success message or is redirected back to the Profile screen with updated data. 4. Backend confirms that user data has been updated successfully.
Actual Results:	Input accepted, data saved, and profile updated successfully.
Post Conditions:	User's profile data is updated and saved in the backend database
Status: (Pass/Fail)	Pass
Other Comments:	Form validation, save button functionality, and backend update process all worked as expected.

3.3 Conclusion:

At the end of the testing phase, the Car Deposit Platform demonstrated satisfactory performance, fulfilling its intended functionality across all executed test cases. These covered key features including User Signup, Authentication (Login/Logout), Session Security, Vehicle Listing Upload with multimedia integration, Real-Time WebSocket Bidding Rooms, Balance Verification Controls, Admin Moderation Panels, and Automated Audit Report Generation.

All test cases passed successfully, with actual results matching expected outcomes. No major bugs, structural crashes, or racing conditions were observed during simulated concurrent load testing, and WebSocket events, database API responses, route guards, and live dashboard metrics transitioned smoothly. The transactional data flow—particularly from user onboarding and live streaming bidding loops to final asset settlement and reporting—remained highly consistent and secure.

The platform accurately managed real-time streaming bid increments, validated bids against individual wallet ledger entries, instantly broadcasted peak active bids globally to all listening clients, and successfully compiled structured, downloadable transaction audit sheets in PDF format—all verified during rigorous testing cycles.

In terms of UI/UX, the web application offers a clean, modern, and ultra-responsive interface with high-fidelity visualization cards and accessible navigational blocks designed for a premium user base. All data capture forms include strict client and server-side model validations along with clear feedback alerts, significantly mitigating user input friction and securing database integrity.

Overall, the platform is stable, structurally resilient under high concurrency demands, and perfectly aligns with the engineering and usability objectives set during the system planning phases. With rigorous unit validations, cascade integrity checks, and WebSocket performance calibrations completed, the Car Deposit Platform is now fully functional and primed for scalable deployment in the digital automotive auction marketplace.

Chapter 4 - USER MANUAL

User Authentication and Log In

This section guides you through the process of accessing your account on the Car Deposit Platform. Secure authentication ensures that your active bidding sessions, wallet balance, and car listings remain protected.

- If you are a new user, you can switch tabs by clicking **Register** at the top of the white container box, or by clicking the "**Register Here**" link located right below the main heading.

Figure 12 shows the Login page. The page features a light blue header with the title "Login" and a breadcrumb "Home > Login". Below the header is a white container box. Inside the container, there are two tabs: "Register" and "Login". The "Login" tab is active. Below the tabs, there is a "Log In" heading, a link "New member? Register Here", and two input fields: "Enter Your Email*" and "Password*". There is a "Remember me" checkbox and a "Forgot Password" link. A blue "Log in" button is at the bottom of the form. Below the button, it says "Or Register With" followed by Google and Apple icons.

Figure 12 Singing in

Step-by-Step Log In Instructions

To access your personal buyer, seller, or administrator dashboard, follow these simple steps:

1. Enter Your Registered Email:

- Click inside the field labeled "**Enter Your Email**".
- Type your full registered email address (e.g., user@example.com).
- *Note:* This field is required and will validate that the input follows a standard email format.

2. Enter Your Password:

- Click inside the field labeled "**Password**".
- Type your account password carefully.
- **Security Feature:** By default, your characters will be masked (hidden) as dots for privacy. If you want to double-check what you typed, click the **Eye Icon** (Show/Hide toggle) on the far right edge of the password box to reveal the text.

User Registration (Sign Up)

Register

Home > Register

Register Login

Register

Do you already have an account? [Login Here](#)

Personal Information

Enter Your Full Name*

Enter Your Email* Enter Mobile Number*

India (+91) ✓

Account Information

Username*

Check Availability

Password* Confirm Password*

Prove You Are Human

☐ I'm not a robot

☐ I agree to the Terms & Conditions.

Create Account

Or Login With

Figure 13 Singing up

Step-by-Step Registration Instructions

To create your account, navigate to the **Register** tab and fill out the form using the following steps:

1. Personal Information

- **Enter Your Full Name*:** Type your legal first and last name.
- **Enter Your Email*:** Provide a valid email address. This will be used for account notifications, JWT session tracking, and password recovery.
- **Enter Mobile Number*:** Select your country code from the dropdown menu and type your active phone number.

2. Account Information

- **Username*:** Choose a unique username for your profile. You can click the "**Check Availability**" link below the field to ensure no other member is using it.
- **Password* & Confirm Password*:** Type a strong password, then re-enter it exactly in the confirmation box to ensure there are no typos.

3. Security & Verification

- **Prove You Are Human:** Check the "**I'm not a robot**" reCAPTCHA box. This security measure prevents automated malicious bots from creating fake accounts.
- **Terms & Conditions:** Read through the platform guidelines and check the box next to "**I agree to the Terms & Conditions**" to proceed.

4. Submit Form

- Click the dark blue "**Create Account**" button. The system will securely hash your password, commit your details to the database, issue a secure session token, and redirect you to your main dashboard.

Alternative Quick Registration (Social Sign-Up)

Instead of filling out the manual form, you can create an account instantly using your existing social profiles under the "**Or Login With**" section:

- **Google:** Click the Google icon to link your account profile.
- **GitHub:** Click the GitHub icon to link your developer profile.

Home Page Interface

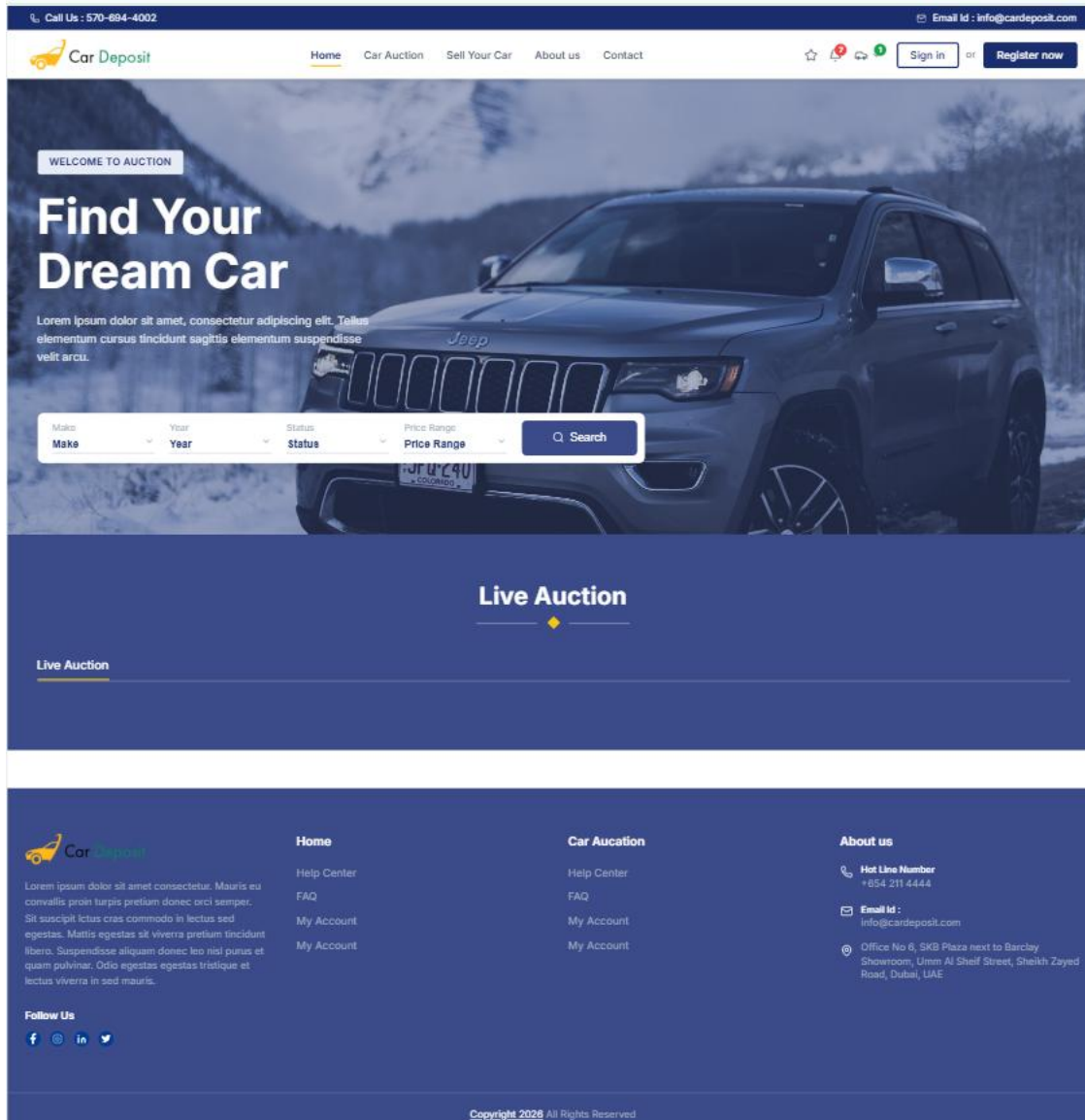


Figure 14 Home

Car Auction Catalog & Search Filter Interface

The Car Auction page is the primary marketplace viewport where users browse active or concluded vehicle lots, apply highly granular search parameters, and initiate the bidding process.

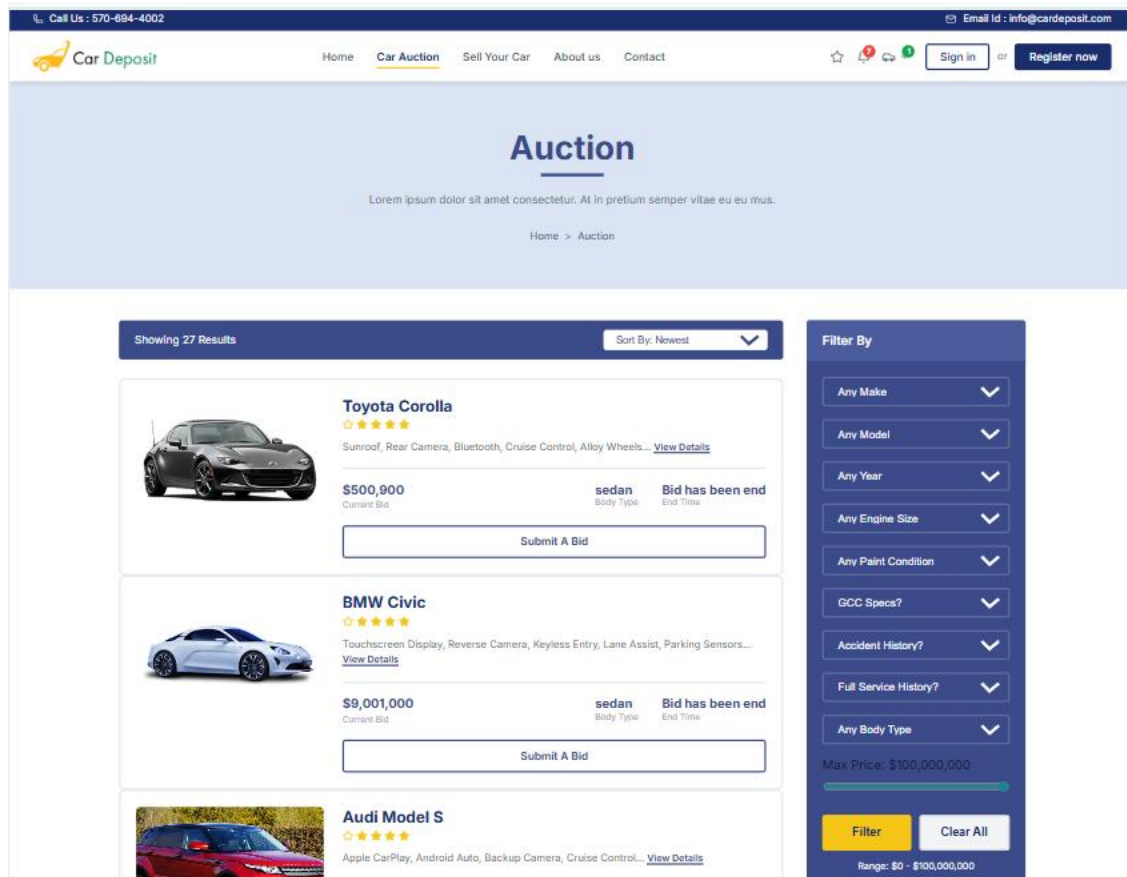


Figure 15 Car Auction

Key UI Components & Navigation Controls

The layout is structurally divided into two main panels: a central **Auction Catalog Feed** and a right-side **Advanced Filtering Sidebar**.

1. Marketplace Information Header

- **Breadcrumb Navigation:** Displays Home > Auction to help users track their location within the application directory.
- **Results Counter:** Displays the live number of active entries currently matching the database query (e.g., "Showing 27 Results").

- **Sorting Dropdown Menu:** Allows users to dynamically change the order of items displayed. Options include sorting by `Newest` listings, price trends, or ending times.

2. Auction Listing Feed Cards

Each vehicle is encapsulated in a dedicated product row detailing its metadata pulled from the database:

- **Vehicle Identity & Visual Asset:** Shows the vehicle image alongside its structural title (e.g., *Toyota Corolla*, *BMW Civic*), rating stars, and short technical descriptions with a **"View Details"** anchor link.
- **Pricing Elements:** Displays the **Current Bid** allocation dynamically updated by the websocket gateway layer.
- **Lot Properties:** Displays key metadata values such as **Body Type** (e.g., *sedan*) and **End Time** tracking. If an auction has reached its time limit, it flags a prominent *"Bid has been end"* notification.
- **Call-to-Action (CTA) Button:** Features a wide **"Submit A Bid"** action trigger that opens the real-time live auction room for that specific vehicle layout.

3. Advanced Filter Sidebar ("Filter By")

An expansive structural control interface allowing users to filter database records instantly based on deep criteria matching:

- **Dropdown Selection Filters:** * `Any Make / Any Model / Any Year`: Standard manufacturer identifiers.
 - `Any Engine Size / Any Paint Condition / Any Body Type`: Spec-specific attributes.
 - `GCC Specs? / Accident History? / Full Service History?`: Critical structural risk variables for collectors.
- **Max Price Slider Component:** A interactive slider component allowing users to set exact budget limits (ranging from \$0 to \$100,000,000).
- **Execution Actions:**
 - **Filter Button:** Click the solid yellow button to commit selected variables and refresh the catalog stream.
 - **Clear All Button:** Resets all fields to null states and pulls the default global catalog database views.

Vehicle Submission Interface ("Sell Your Car")

The "Sell Your Car" page is a comprehensive submission portal that allows authorized sellers and dealers to list their vehicles for upcoming live auctions. This multi-sectional form ensures all

necessary seller validation and granular vehicle metrics are gathered before an auction lot is approved.

Sell Your Car

Lorem ipsum dolor sit amet consectetur. Ali in pretium semper vitae eu eu mus.

[Home](#) > [Sell Your Car](#)

Tell us about your car

Please give us some basic about yourself and car you'll like to sell. We'll also need details about the car's title status as well as 50 photos that highlight the car's exterior and interior condition.

We'll respond to your application within a business day, and we work with you to build a custom and professional listing and get the auction live.

Your Info

Dealer or Private party?

☒ Dealer
 ☐ Private party

First name*

Last name*

Email*

Phone number*

PK (02) ▼

Car Details

VIN*

Year*

Select Year ▼

Make*

Select Make ▼

Model*

All Models ▼

Mileage (in miles)

Engine size

Select ▼

Color*

Select ▼

Has GCC Specs

Select ▼

Noteworthy options/Features

Accident History

Select ▼

Full Service History

Select ▼

Has the car been modified?

☒ Completely stock
 ☐ Modified

Starting Bid / Base Price*

Auction End Date*

02/06/2022

📅

Auction End Time*

--:--:--

🕒

Upload Photos

Add Photos (0 selected)

Submit

Figure 16 Add Car

Live Vehicle Auction Detail Interface

The Vehicle Auction Detail page is the interactive, real-time core of the platform where logged-in buyers monitor active or closed bidding streams, evaluate vehicle specifications, verify seller background data, and place live bids.

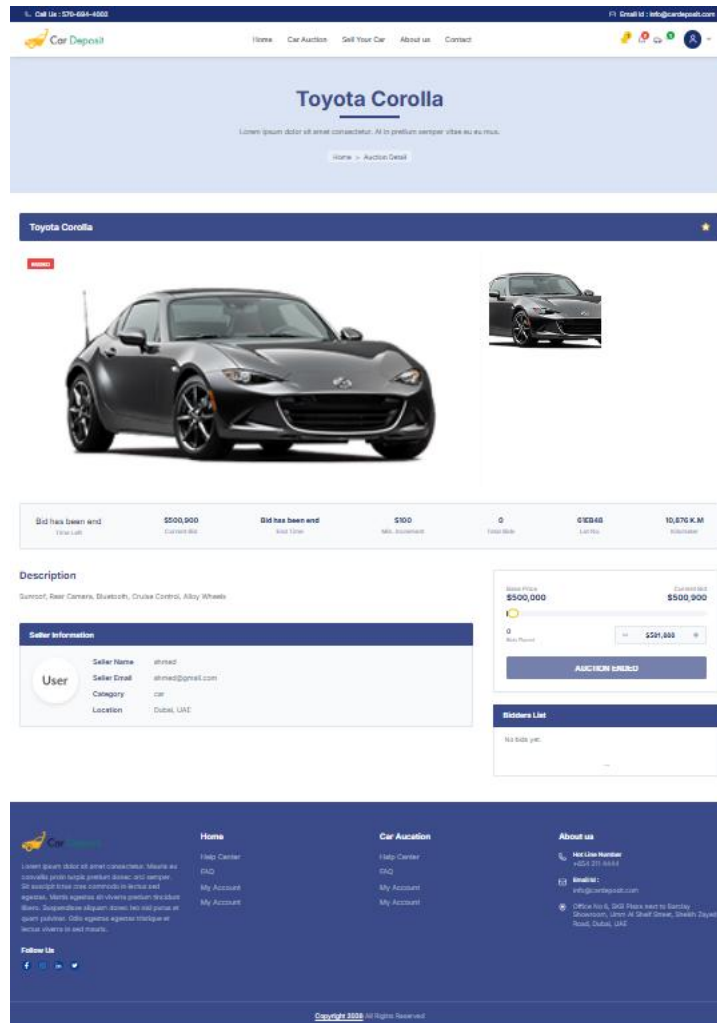


Figure 17 Car Auction Detail

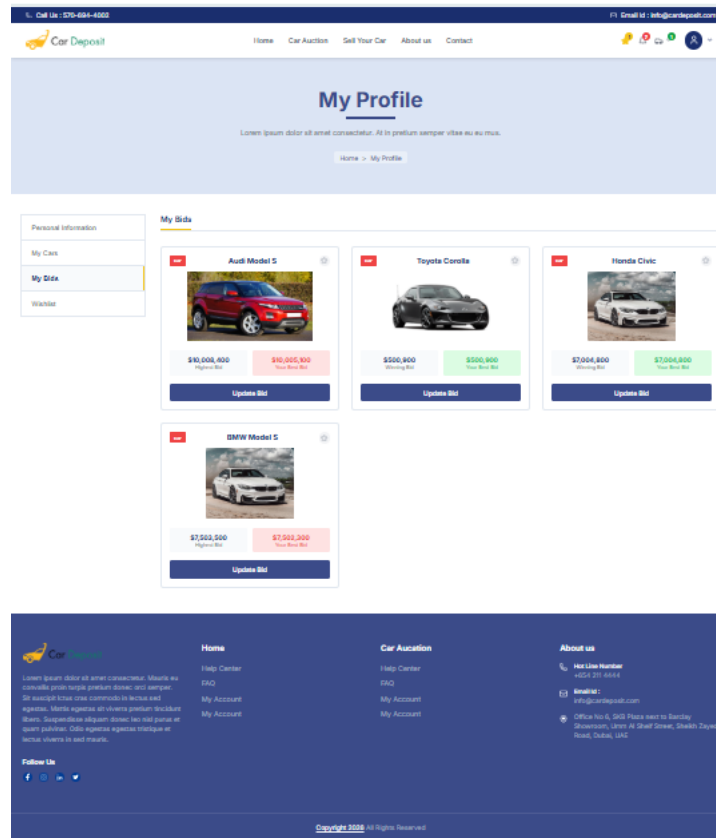


Figure 18 Profile

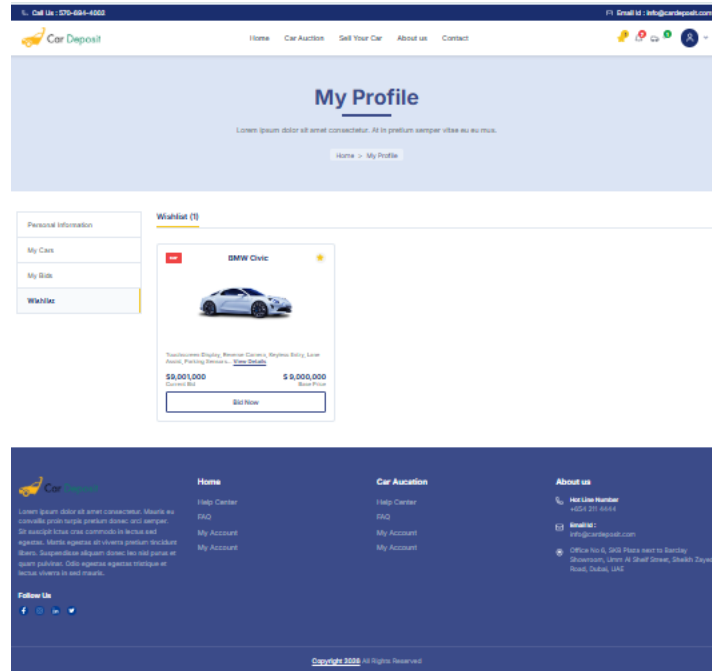


Figure 19 Report

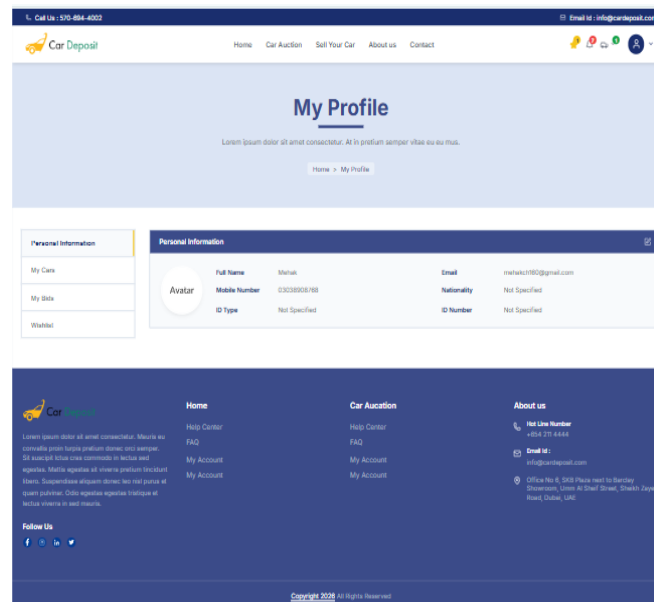


Figure 20 Edit Profile

References

- [1] M. Fowler, *Patterns of Enterprise Application Architecture*. Boston, MA, USA: Addison-Wesley, 2003. (*Reference for Three-Tier and Layered Architecture*)
- [2] M. B. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, Internet Engineering Task Force (IETF), May 2015. Available: <https://datatracker.ietf.org/doc/html/rfc7519>. (*Reference for Secure Registration and Login Sessions*)
- [3] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA, USA: Addison-Wesley, 1994. (*Reference for Publisher-Subscriber Patterns used in WebSocket Event Broadcasting*)
- [4] UAF Department of Computer Science. (2025). Guidelines & Report Format for FYP Documentation. University of Agriculture, Faisalabad.
- [5] I. Fette and A. Melnikov, "The WebSocket Protocol," RFC 6455, Internet Engineering Task Force (IETF), Dec. 2011. Available: <https://datatracker.ietf.org/doc/html/rfc6455>. (*Reference for Real-Time Live Bidding Sockets*)

Follow UAF reference style to cite book, research paper (journal as well as conference paper) and website references. At least 3 to 4 reference must be present in Chapter 1 and 2.

Formatting instructions:

Following formatting is already applied on the document. However, it is explicitly mentioned below:

- Minimum length of the Report: 30 pages
- Font style : Times New Roman
- Paragraph font size: 12pt
- Main Heading Size: 14pt + Bold (before and after spacing 12pt)
- Sub Heading Size: 13pt + Bold (before and after spacing 8pt)
- Sub sub heading size 12pt + bold (before and after spacing 6pt)
- Paragraph Alignment: Justified
- Picture/Chart Alignment: Center
- Picture/chart/table heading font: Times New Roman
- Picture/chart/table heading font size: 10pt, Italic, center alignment
- Picture caption goes under the picture without any extra line and line space
- Table caption goes above the table without any extra line and line space
- Table heading: Times New Roman, 10pt, Bold
- Table text: Times New Roman, 10pt
- Line Spacing: 1.15
- Left/Right/Top/Bottom Margins: 1 inch
- Table of Contents, List of Tables, List of Figures Heading: Times New Roman. 14pt, Bold
- Table of Contents, List of Tables, List of Figures: Times New Roman. 12pt
- For Table of Contents, List of Tables & List of Figures use Roman number as page number format in footer (center align)
- For Introduction onwards, use integer number as page number format in footer (center align)

For Submission:

Three hard binded copies of technical report are required with memory card (attached with each copy) containing technical report and project (exe file + all source code).